PeerWatch: Multi-Signal Network Anomaly Detection for Local Area Networks

Darius Chitoroaga

MEng Computer Science

Submission date: TODO: Day Month Year

Supervised by: TODO: Supervisor's Name

Disclaimer: This report is submitted as part requirement for the MEng Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text.

The report may be freely copied and distributed provided the source is explicitly acknowledged.

CONTENTS

1 Introduction ... 3
1.1 Aims and Goals ... 5
1.2 Scope ... 6
1.3 Approach ... 6
1.4 Report Structure ... 7
2 Background and Context ... 7
2.1 LAN-Layer Attack Techniques ... 7
2.1.1 ARP Cache Poisoning ... 7
2.1.2 Device Substitution Without ARP Forgery ... 8
2.1.3 Service-Level Impersonation ... 8
2.1.4 Coordinated Fleet Attacks ... 9
2.1.5 Evasion and the Attacker's Observable Constraint ... 9
2.2 Device Fingerprinting ... 10
2.2.1 Active Fingerprinting ... 10
2.2.2 Passive Fingerprinting ... 10
2.2.3 Cryptographic Identity Anchors ... 11
2.2.4 Route Stability and MAC OUI Correlation ... 12
2.3 Related Work and Tool Survey ... 12
2.3.1 Enterprise Network Detection and Response ... 12
2.3.2 Network Intrusion Detection Systems (NIDS) ... 12
2.3.3 Single-Signal Lightweight Tools ... 13
2.3.4 Academic Work on Device Fingerprinting ... 13
2.3.5 LLM-Assisted Security Triage ... 14
2.4 Tools and Libraries ... 14
2.4.1 Active Scanning: nmap ... 14
2.4.2 Passive Capture: Scapy ... 15
2.4.3 LLM Integration: Ollama, Phi-4, and LangChain ... 15
2.4.4 Configuration and Validation: Pydantic ... 16
2.5 Evidence Accumulation and Anomaly Scoring ... 16
2.5.1 Weighted Scoring ... 17
2.5.2 Signal Compositing ... 17
2.5.3 Exponential Decay and the Cold-Start Period ... 17
2.5.4 Threshold Calibration ... 18
3 Requirements and Analysis ... 18
3.1 Detailed Problem Statement ... 18
3.2 Requirements ... 20
3.2.1 Functional Requirements ... 20
3.2.2 Non-Functional Requirements ... 21
3.3 Use Cases ... 22
3.4 Analysis ... 23
3.4.1 Data Model ... 23
3.4.2 Component Identification ... 24

3.4.3 Key Design Decisions ... 25
4 Design and Implementation ... 26
4.1 System Architecture ... 26
4.2 Scan Loop and Data Ingestion ... 27
4.2.1 nmap Invocation ... 27
4.2.2 XML to JSON Conversion and Field Stripping ... 28
4.2.3 The Injection Interface ... 28
4.2.4 NormalisedData and the os_candidates Design ... 28
4.2.5 Ingestion Tracking, Rate Limiting, and Shutdown ... 29
4.3 Device Identity Store and Fingerprint Comparison ... 29
4.3.1 Peer Model and Identity Keying ... 30
4.3.2 Fingerprint Comparison Engine ... 30
4.3.3 False-Positive Suppression ... 31
4.3.4 Scoring and Exponential Decay ... 32
4.3.5 Conflict Resolution, Volatile Peers, and Persistence ... 32
4.4 Passive Observation Layer ... 33
4.4.1 Architecture and Testability ... 33
4.4.2 Per-Signal Capture ... 34
4.4.3 Route Tracking and ASN Enrichment ... 35
4.5 Fleet Correlation ... 36
4.5.1 Tick Window and Event Collection ... 36
4.5.2 Pattern Matching, Boost Cap, and Fleet Context ... 37
4.6 LLM-Assisted Investigation ... 38
4.6.1 Investigation Pipeline ... 38
4.6.2 Prompt Design and Injection Prevention ... 38
4.6.3 LLM Integration, Output Schema, and Fallback ... 39
4.6.4 Automated Follow-up Scans and Cryptographic Anchors ... 39
4.7 Autonomous Remediation ... 40
4.7.1 Guard Chain and Enforcement Modes ... 40
4.7.2 Block TTL, Audit Log, and Atomic Rewrite ... 41
4.8 Configuration and Extensibility ... 42
References ... 43
E Artifact Appendix ... 46

Abstract —

1 INTRODUCTION

The typical home network today bears little resemblance to the isolated subnet of a decade ago. Driven by the proliferation of connected devices, residential and small-office subnets now carry a mixture of enterprise endpoints, smart home appliances, voice assistants, and networked infrastructure that receives infrequent security updates and is rarely under active administrative oversight

[6]. The 2016 Mirai botnet demonstrated at scale that consumer-grade connected devices could be silently weaponised for coordinated global attacks without their owners' awareness [1]; the attack surface in the average home network has only expanded since. Yet the monitoring tools available to independent administrators have not kept pace with this expanded threat surface, and this gap motivates the present work.

Local area networks are built on protocols designed for trusted environments. The Address Resolution Protocol (ARP), which maps IP addresses to MAC addresses on a subnet, carries no authentication: any host can broadcast an ARP reply claiming any binding it chooses [25]. An attacker with network access can silently impersonate another device, redirecting traffic through a machine they control; a man-in-the-middle (MitM) attack that enables credential theft, session hijacking, and traffic inspection without alerting the victim or the network administrator [4].

The standard hardware-enforced defences against ARP spoofing: Dynamic ARP Inspection (DAI) and DHCP snooping, require managed layer-2 switches that maintain a trusted binding table [26]. These features are absent from commodity home and small-office routers, the environments where monitoring is often most needed and least configured. Software-based tools such as arpwatch [16] fill part of this gap by alerting on changes to ARP bindings, but they operate on a single signal. An attacker who compromises a device without forging ARP replies, physically replaces a device, or gradually migrates a service to a new host is entirely invisible to ARP-only monitors.

A more robust approach treats device identity as a composite of independent signals: the operating system inferred from TCP/IP stack behaviour, the set of open ports and running services, the time-to-live (TTL) field of outgoing packets, the route traffic takes through the network, and the cryptographic identity anchors that services such as SSH and TLS expose. Significant drift in any of these signals, or the co-occurrence of smaller drifts across several signals in the same scan cycle, indicates that the device behind an IP address may not be the device previously observed there. When the same anomaly pattern appears simultaneously across multiple hosts, the probability of a coordinated attack rather than a routine configuration change rises substantially. The challenge is calibrating this without drowning operators in noise. Routine events, a firmware update that alters a device's OS fingerprint, a DHCP lease renewal that reassigns an IP, or a router reconfiguration that shifts TTL values, produce observations indistinguishable from an attack in isolation. A monitor that fires on every anomaly is quickly ignored, defeating its own purpose, causing alertfatigue for admins, [14]; careful evidence accumulation, signal weighting, and decay are therefore as important as the signals themselves.

Enterprise network detection and response (NDR) platforms such as Darktrace [9] and Cisco Secure Network Analytics [7] provide sophisticated multi-signal monitoring, but require dedicated appliances, managed infrastructure, or cloud connectivity that place them beyond the reach of home and small-office administrators. Lightweight open-source tools such as arpwatch are deployable on existing hardware but, as noted, are restricted to a single signal. Network intrusion detection systems (NIDS) such as Snort [28] and Suricata [22], and the network analysis framework Zeek [24], offer deep packet inspection and protocol analysis but operate on signature matching rather than per-device identity drift; they are designed for dedicated network taps and do not maintain a persis-

tent fingerprint model per LAN device. The gap between these categories of tool is summarised in Table 1. No open-source, locally-deployable tool currently integrates active device fingerprinting, passive observation, and fleet-level co-occurrence analysis in a single monitor.

Tool	Active scan	Passive capture	Fleet correl.	Local deploy
Darktrace / Cisco SNA	✓	✓	✓	✗
Snort / Suricata	✗	✓	✗	✓
Zeek	✗	✓	✗	✓
arpwatch	✗	✓ (ARP only)	✗	✓
PeerWatch	✓	✓	✓	✓

Table 1: Detection capability comparison across monitoring tools relevant to small-office administrators.

This project aims to close that gap, building a tool designed specifically for the independent administrator who has neither managed switching infrastructure nor an enterprise security budget.

The threat model assumed throughout this work places the attacker within the local network — through a compromised device, a rogue wireless association, or physical access — and grants them the ability to forge ARP replies, substitute physical devices, or redirect services to attacker-controlled hosts. The attacker is not assumed to perfectly replicate every observable property of the target; the detection strategy exploits precisely this constraint. The monitoring host is assumed to hold a network interface capable of promiscuous-mode capture and sufficient privilege to run periodic `nmap` scans, and is itself assumed to be uncompromised.

1.1 Aims and Goals

The *aim* of this project is to design, implement, and evaluate **PeerWatch**: an open-source, autonomous network anomaly detection daemon that fingerprints subnet devices across multiple independent signals, tracks device identity over time, and flags evidence of spoofing or coordinated attack; deployable on hardware a home or small-office administrator already owns, with no cloud dependencies.

The specific *goals* are:

1. **Active device fingerprinting.** Build a periodic `nmap`-based scan loop [17] that maintains a persistent identity store (`PeerStore`) for each discovered device, tracking OS family, open ports, service types, and MAC address. Define a scoring engine that accumulates suspicion for detected drift between successive scans and decays it exponentially, so stale anomalies do not permanently penalise a device.

2. **Passive monitoring layer.** Supplement active scans with live packet capture that tracks TTL consistency, monitors ARP replies for binding conflicts, performs TCP stack fingerprinting [32],

and checks route path stability; detection signals that require no active probing and that catch attacks leaving no nmap-visible trace.

3. **Fleet-level correlation.** Implement a correlator that identifies coordinated anomaly patterns across multiple peers within the same scan tick, such as simultaneous ARP poisoning of several hosts or a subnet-wide route shift, and scores them independently of per-device analysis.

4. **LLM-assisted triage.** Integrate a locally-run large language model that produces structured natural-language investigation reports when a device's suspicion score crosses a threshold, reducing the cognitive burden on the operator who must decide whether to act [14]. Recent advances in locally-runnable models and software allowing models to run on commodity hardware via Ollama [21] have made LLM-assisted triage increasingly viable in isolated environments without cloud infrastructure. No monitored traffic leaves the subnet, no API key or subscription is required, and the system remains functional in air-gapped deployments. The LLM explains triggered alerts and does not participate in the detection decision itself. To prevent indirect prompt injection [13], the model is never given access to raw peer data such as packet payloads, hostnames, or service banners; it receives only structured summaries produced by the detection pipeline, containing numeric scores, event type labels, and device identifiers.

5. **Empirical evaluation.** Measure detection performance across a suite of simulated attack scenarios drawn from MITRE ATT&CK and CVE-documented techniques, and characterise the false positive rate under clean traffic.

1.2 Scope

PeerWatch targets subnets of up to approximately 254 hosts (/24 CIDR blocks) on hardware the administrator already owns: a dedicated monitoring device, repurposed laptop, or single-board computer. The following are explicitly outside scope: WAN-level or inter-subnet threats; decryption of encrypted traffic or deep protocol reconstruction; scalability to multi-subnet enterprise deployments; and detection of attacks originating from the monitoring host itself. The system is designed for deployments where the operator is also the administrator, home users and small offices, not for SOC environments where dedicated appliances and analyst teams are available.

1.3 Approach

PeerWatch was developed iteratively across three phases, each adding a new detection dimension while preserving the correctness of the previous ones. A key insight shaping the architecture was that each phase was motivated by a concrete category of attack the previous layer could not detect: ARP-only monitors are blind to device substitution that bypasses ARP entirely, and active scan profiles alone cannot catch adversaries who replicate nmap-visible behaviour while diverging at the packet level. The interaction between signals also proved more informative than any individual signal, simultaneous drift in OS fingerprint, TTL baseline, and port profile within one scan tick is qualitatively different from any single drift in isolation.

Phase 1 established the core detection loop: periodic `nmap` scans of the target subnet, a `PeerStore` keyed by MAC address (or IP for MAC-less devices), and a rule-based scoring engine that compares

each scan against the stored baseline. When a device's score crosses a configurable threshold, a `SuspiciousAgent` invokes a locally-run Ollama model for structured JSON triage, falling back to rule-based severity assignment if the model is unavailable.

Phase 2 added a passive capture loop running alongside the active scan. Passive observations: TTL deviation from an established baseline, ARP reply conflicts, TCP fingerprint inconsistencies, IP ID sequence jumps, and traceroute path changes, accumulate in per-device observation models and contribute to the suspicion score independently of the active scan. This layer catches attacks that exactly mimic a target's nmap-visible profile but cannot replicate every low-level TCP stack behaviour.

Phase 3 introduced the `FleetCorrelator`, which runs each tick after ingestion and before investigation. It detects patterns where multiple peers exhibit the same anomaly simultaneously, applies a configurable suspicion boost to each matching device, and injects fleet-level context into the LLM prompt, so the agent can reason about coordinated attacks rather than isolated per-device events. Phase 3 also added SSH host-key and TLS certificate fingerprint tracking as high-value cryptographic identity anchors.

Throughout, all scoring weights, thresholds, and daemon settings are exposed through a single Pydantic [8] configuration model, keeping the pipeline transparent and tunable without code changes. The system is designed for subnets of a few hundred hosts on hardware a home or small-office administrator already owns; larger deployments would require scan parallelism.

1.4 Report Structure

Chapter 2 provides the context for this work: background on LAN-layer attack techniques, a survey of related detection tools, and an introduction to the key libraries and components on which PeerWatch is built. Chapter 3 captures the requirements and analyses them into an initial design. Chapter 4 describes the design and implementation of PeerWatch in detail, following the three-phase structure above. Chapter 5 presents the testing strategy and empirical evaluation across the simulated attack suite and a clean-traffic false-positive measurement. Chapter 6 concludes with a critical assessment of the results against the goals set out above, and directions for future work.

2 BACKGROUND AND CONTEXT

2.1 LAN-Layer Attack Techniques

Understanding the threat landscape PeerWatch targets requires examining how attackers exploit the trust assumptions embedded in local network protocols. The attacks described below are real-world examples: each maps to documented CVEs or MITRE ATT&CK techniques [20] and corresponds directly to detection scenarios evaluated in Chapter 5.

2.1.1 ARP Cache Poisoning

The Address Resolution Protocol's trust model was introduced in Chapter 1; this section examines the attack mechanics in sufficient detail to motivate the detection signals that follow.

An ARP poisoning attack proceeds in two steps. The attacker broadcasts gratuitous ARP replies (unsolicited announcements claiming a particular IP-to-MAC binding) targeting both the victim host and the default gateway. Because standard operating system ARP implementations update their cache on receipt of any ARP reply regardless of whether a request was issued [25], both targets update their routing tables to direct traffic through the attacker's machine. The attacker, forwarding traffic between victim and gateway while inspecting it, achieves a man-in-the-middle position transparent to both parties.

From this position an attacker can intercept cleartext credentials, inject content into unencrypted sessions, or strip TLS by presenting a forged certificate to the victim while maintaining a legitimate upstream connection. CVE-2020-25705 (SAD DNS [18]) demonstrates that IP address ownership cannot be treated as self-authenticating even without ARP manipulation; ARP poisoning is the local-network equivalent, documented as MITRE ATT&CK T1557.002 [20].

The MAC address change ARP poisoning produces is detectable by tools such as arpwatch. It is only one observable, however: if the attacker substitutes a device with an identical MAC, or if the attack proceeds through a compromised device that retains its original MAC; the ARP signal is entirely clean, and a richer set of observations is required.

2.1.2 Device Substitution Without ARP Forgery

A more operationally sophisticated attack replaces or impersonates a device without disturbing ARP bindings. The attacker clones the target device's MAC address, trivially accomplished on modern operating systems, and begins responding to network traffic from attacker-controlled hardware. No ARP anomaly is produced; arpwatch observes nothing.

What does change is the device's observable identity at the network stack level. Different hardware exposes a different TCP/IP stack fingerprint: initial TTL values, IP identification field patterns, TCP window size, window scaling options, and initial sequence number generation are determined by the operating system and kernel version, not the MAC address [17]. An attacker running Windows on hardware that previously hosted a Linux IoT device will produce OS fingerprinting results inconsistent with the stored baseline, a port profile that may differ, and a TTL baseline that diverges from the established per-device average.

MAC spoofing without OS replication is documented as T1564.006 [20]. The limitations of MAC-only monitoring were demonstrated in CVE-2004-0699 [20], where device identity assumptions were exploited precisely because address-layer credentials were not backed by any deeper identity verification.

2.1.3 Service-Level Impersonation

Where device substitution attacks target the network identity of a host, service-level attacks target specific services running on it.

Service backdoors and protocol takeover. An attacker who has compromised a device may install additional services, replace existing ones with attacker-controlled versions, or expose services on non-standard ports. These produce observable port profile drift: new ports appear in subsequent scans, or services on known ports change type. T1543 (Create or Modify System Process)

[20] and US-CERT advisory TA17-181A [30] document service persistence techniques used in documented intrusions. More recent examples include CVE-2024-6387 (regreSSHion [27]), a remote code execution vulnerability in OpenSSH that could enable replacement of a running SSH daemon, and CVE-2024-4577, where protocol-level service replacement produced detectable port/protocol mismatches.

Cryptographic identity anchors. SSH servers present a host key fingerprint to clients on each connection; TLS servers present a certificate fingerprint. These are high-entropy, asymmetrically generated values that cannot be forged without the corresponding private key. A device substitution that replaces an SSH or HTTPS server necessarily exposes a different fingerprint to any observer who has recorded the original. This makes host key and certificate tracking qualitatively stronger evidence than heuristic signals: a determined attacker can tune kernel parameters to approximate an OS fingerprint, but presenting a valid SSH host key for a device they do not control is computationally infeasible.

2.1.4 Coordinated Fleet Attacks

The attack patterns above are considered in isolation; in practice, targeted intrusions frequently affect multiple hosts in the same network concurrently. The distinction matters for detection: a single device showing OS fingerprint drift is consistent with a firmware update; four devices showing simultaneous OS drift is not.

Fleet-level patterns observable in real intrusions include gateway substitution campaigns, where the attacker poisons ARP caches across multiple hosts simultaneously to redirect all subnet traffic, subnet-wide route shifts produced by compromising the default gateway, and identity sweeps where multiple devices are replaced in the same window to reduce single-device anomaly visibility. The 2016 Mirai botnet [1] is a documented precedent for coordinated compromise of multiple LAN devices; post-compromise behaviour included systematic service port exposure consistent with the service sweep pattern.

A critical property of coordinated attacks is that per-device analysis may silently miss them: if each individual device scores below the investigation threshold, no alert fires. Fleet-level correlation (detecting that several peers exhibit the same anomaly class within the same observation window) is the mechanism that closes this gap, and is described in detail in Section 4.x.

2.1.5 Evasion and the Attacker's Observable Constraint

The detection strategy throughout PeerWatch rests on the assumption stated in Chapter 1: an attacker cannot perfectly replicate every observable property of the device they are impersonating. This section examines where that assumption is strong and where it is not.

A determined attacker can replicate several signals. Port profile can be matched by opening the same ports; service type labels can be mimicked by running compatible stacks; OS fingerprint can be approximated through kernel parameter tuning — adjusting TCP initial TTL, window size, and timestamp options. Scenario I in the evaluation (T1036.004 masquerading, CVE-2024-3094) demonstrates this boundary: an attacker who carefully mirrors the target's service profile produces a suspicion score of 0.0, a correctly identified limitation of the active-scanning layer.

What is structurally hard to replicate: TTL baselines accumulated over many scan cycles, where hardware-level routing differences compound; TCP/IP stack microvariances below the granularity of kernel parameter tuning; IP identification field counter patterns, which reflect packet generation rate and hardware; and cryptographic identity anchors, where forging a host key or certificate requires possession of the private key.

The practical consequence is that detection reliability increases with the number of independent signals covered. An attacker who evades the nmap OS fingerprint signal may still expose TTL drift; one who suppresses TTL drift may expose a host key change. No single evasion technique neutralises all signals simultaneously without perfectly replicating the target device; which is, by definition, device substitution rather than impersonation.

2.2 Device Fingerprinting

Device fingerprinting derives a stable identity for a networked host from observable protocol behaviour, without relying on self-reported credentials. This section surveys the techniques PeerWatch draws on, their mechanisms, and their limitations. Chapter 4 describes how each is implemented within the pipeline.

2.2.1 Active Fingerprinting

Active fingerprinting sends crafted probe packets to a target and analyses the responses. Nmap's OS detection engine [17] sends a battery of TCP, UDP, and ICMP probes designed to elicit responses that vary by operating system implementation: a SYN to an open port, a SYN to a closed port, a FIN to an open port, and several ICMP variants, each exposing differences in TCP initial sequence number generation, window size and scaling, timestamp option presence, don't-fragment bit handling, and IP identification field behaviour. The response profile is matched against a database of approximately 5,000 OS fingerprints; a confident match returns the OS family and version, a low-confidence match returns the closest candidates with a probability score.

Beyond OS family, the set of open ports constitutes a stable per-device identity vector. Jaccard similarity: the size of the intersection divided by the size of the union of two port sets, quantifies drift between successive scans. A score below a configurable threshold (0.6 by default) flags meaningful port profile change. Service version detection supplements this: nmap's banner-grabbing identifies the service type on each open port, catching cases where the port number is unchanged but the responding application is not.

Active fingerprinting has well-known limitations. Root privilege is required to send raw packets; probe traffic is visible to network monitors; and OS detection accuracy degrades in virtualised or containerised environments where the guest stack is partially masked by the hypervisor. Most importantly for this work, OS fingerprint can be partially spoofed through kernel parameter tuning, adjusting TCP initial TTL, window size, and timestamp options, as established in Section 2.1.5. These limitations motivate the passive layer described next.

2.2.2 Passive Fingerprinting

Passive fingerprinting derives OS and behavioural signals from traffic already on the wire, without sending probes. The p0f tool [32] pioneered this approach: SYN packets from TCP initiators expose

the sender's TTL, TCP window size, window scaling factor, maximum segment size, timestamp echo, and IP option fields: a profile that differs reliably across OS families without any active interaction.

The most operationally significant passive signal for per-device identity tracking is the TTL baseline. Each OS family initialises outgoing packets with a characteristic TTL (Linux: 64, Windows: 128, Cisco IOS: 255); after traversing a fixed number of hops to the monitoring host, the observed value is stable for a given device. A single TTL observation is unreliable, routing changes and transient variation exist, but a per-device baseline accumulated over many observations is robust. Deviation of more than 15 from the established baseline is a reliable indicator of either a routing path change or a device substitution.

The IP identification field provides a complementary signal. Older Linux kernels and many embedded devices generate sequential IP ID values, producing a monotonically increasing counter visible in captured packets. Modern Linux and Windows randomise the field. A transition from sequential to random generation, or a counter jump inconsistent with the device's observed packet rate, indicates a stack change.

Passive observation has a structural advantage: it cannot easily be suppressed by an attacker without controlling all outgoing traffic from the compromised device. Its limitation is the inverse: devices generating no observable traffic are invisible, and promiscuous-mode capture requires the monitoring interface to see the relevant frames.

2.2.3 Cryptographic Identity Anchors

Cryptographic identity anchors are high-entropy, asymmetrically generated values that services expose on connection and that cannot be forged without the corresponding private key.

SSH servers generate a host key pair at installation time and present the public key fingerprint to every connecting client. TLS servers present a certificate fingerprint (the hash of the DER-encoded certificate) on each connection. Both values are stable across reboots and software updates; they change only on explicit reconfiguration or reinstallation. A device substitution that replaces an SSH or HTTPS server necessarily exposes a different fingerprint to any observer who recorded the original, making key or certificate change near-certain evidence of server substitution rather than routine maintenance.

The qualitative difference from heuristic signals is significant. OS fingerprint, TTL, and port profile can all be partially spoofed through configuration; presenting a valid SSH host key for a device the attacker does not control requires breaking RSA or ECDSA — computationally infeasible. A single host key change event therefore carries far more evidential weight than a single OS fingerprint drift.

Internet-scale infrastructure monitoring tools such as Censys [10] apply the same principle across the public internet, indexing SSH and TLS fingerprints to track infrastructure changes and detect certificate substitution. PeerWatch applies the technique at LAN granularity, where the baseline is per-device and changes are flagged in real time against a persistent identity store.

2.2.4 Route Stability and MAC OUI Correlation

Route stability. Traceroute maps the sequence of IP hops between the monitoring host and a target device. For a LAN device with a fixed default gateway, this path is constant; a new intermediate hop or changed hop count indicates a routing table manipulation or gateway substitution, consistent with the fleet patterns described in Section 2.1.4. Team Cymru's IP-to-ASN mapping attributes each hop to an autonomous system; a new ASN in the path indicates traffic leaving via an unexpected upstream provider.

MAC OUI correlation. The first three octets of a MAC address identify the hardware vendor registered with the IEEE (the Organizationally Unique Identifier). Cross-checking OUI vendor against nmap-detected OS family catches a class of substitution neither signal alone would flag: an Apple OUI combined with a Linux OS detection indicates the claimed hardware identity is inconsistent with the network stack behaviour.

This signal carries elevated false positive risk. MAC address randomisation, enabled by default in iOS 14+ and Android 10+, assigns locally-administered addresses per network, making OUI attribution unreliable for mobile devices. Virtual machine adapters expose hypervisor vendor OUIs legitimately inconsistent with the guest OS. OUI correlation is therefore treated as a supporting signal rather than a primary detection trigger.

2.3 Related Work and Tool Survey

Section 2.1 established the attack landscape and Section 2.2 the detection signals. This section surveys existing tools and research, explains why each fails to address the gap PeerWatch targets, and positions PeerWatch within the literature. The capability comparison in Table 1 provides a structured summary; this section provides the qualitative analysis behind it.

2.3.1 Enterprise Network Detection and Response

Enterprise NDR platforms represent the most capable end of the monitoring spectrum. Darktrace [9] builds unsupervised machine learning baselines per device, detecting anomalous behaviour relative to each device's individual norm, a conceptual approach similar in spirit to PeerWatch's per-device identity tracking. Cisco Secure Network Analytics [7] analyses NetFlow traffic metadata to detect volumetric and behavioural anomalies across managed infrastructure. Both platforms offer genuine multi-signal detection with documented effectiveness against advanced persistent threats in enterprise environments.

The gap they do not address is deployment model, not technical capability. Darktrace requires cloud connectivity for model updates and investigation tooling, and its detection logic is proprietary and not user-inspectable. Cisco Secure Network Analytics requires Cisco managed switching infrastructure or a cloud licence. Neither platform runs on commodity hardware; neither is available without enterprise licensing. The threat model they address (enterprise perimeter, managed infrastructure, dedicated security operations staff) differs substantially from the one PeerWatch targets.

2.3.2 Network Intrusion Detection Systems (NIDS)

Snort [28] and Suricata [22] are signature-based NIDS that evaluate each packet or flow against a rule set. They are effective at detecting known attack patterns against published rule sets and are widely deployed in enterprise and research networks. Their fundamental limitation here is that they

maintain no per-device identity model: each packet is evaluated in isolation against static rules. Snort can detect ARP spoofing if an ARP rule is deployed, but has no concept of "this device previously ran Windows and now runs Linux", the drift-based detection that characterises PeerWatch's approach.

Zeek [24] is a more flexible protocol analysis framework that supports custom scripting for detection logic. A sufficiently motivated operator could script per-device fingerprint tracking in Zeek, but this is not available out of the box and requires deep domain expertise; Zeek's deployment model also assumes a dedicated high-throughput network tap rather than the edge hardware SOHO (Small Office/Home Office) administrators own. No maintained open-source Zeek module provides the LAN-device identity-drift detection PeerWatch implements.

All three are fundamentally **reactive**: they detect known or statistically anomalous patterns against a current observation. None accumulates a persistent per-device fingerprint baseline that enables temporal identity drift to be measured across scan cycles.

2.3.3 Single-Signal Lightweight Tools

Arpwatch [16] monitors ARP bindings and alerts on new or changed IP-to-MAC mappings. It is lightweight, reliable, and well-suited to the SOHO deployment model. Its limitation is precisely what Section 2.1.2 establishes: it is blind to device substitution that does not disturb ARP bindings, to service-level impersonation, and to coordinated attack patterns that unfold above the ARP layer. XarpS and similar tools share this scope.

Hardware-enforced equivalents, Dynamic ARP Inspection and DHCP snooping [26], are more robust for the ARP signal specifically but require managed layer-2 switching infrastructure absent from SOHO deployments.

Every existing lightweight tool covers a single signal. The design space for a multi-signal, locally-deployable, open-source LAN identity monitor combining active scanning, passive capture, and fleet correlation is effectively vacant.

2.3.4 Academic Work on Device Fingerprinting

A substantial body of academic work addresses device **type** identification from passive network observations. IoT Sentinel [19] identifies IoT device types from DHCP fingerprints and network flow metadata, enabling automatic policy enforcement on home routers. Related work uses DNS query patterns, flow statistics, and protocol behaviour to distinguish device categories for network management and access control.

This work is related but addresses a different question. Classification asks: **what type of device is this?** Identity drift detection asks: **is this the same device as before?** A classifier is not sensitive to identity drift: a compromised device replicating its device type's traffic profile would be correctly classified while going undetected, and a substitution that preserves device type would be invisible. The two approaches are complementary rather than competitive.

Statistical anomaly detection: flow entropy analysis, traffic volume baselines, protocol distribution modelling, underpins commercial NDR systems and has a deep research literature. These operate

at aggregate traffic level, making them sensitive to volumetric shifts but not to low-and-slow device substitution that preserves traffic volume while changing device identity.

Direct prior work on per-device multi-signal identity drift detection in open-source LAN monitoring is, to the author's knowledge, sparse. This gap is the primary motivation for PeerWatch.

2.3.5 LLM-Assisted Security Triage

The application of large language models to security operations has grown rapidly since the emergence of capable instruction-following models. Commercial products including Microsoft Copilot for Security and Google Chronicle AI incorporate LLMs for alert summarisation, threat intelligence synthesis, and first-pass triage of SIEM events, targeting the alert fatigue problem [14] that afflicts security operations teams at scale.

Deploying LLMs in security contexts introduces a specific risk: indirect prompt injection [13]. Where an LLM processes data originating from an adversary controlled source, packet payloads, device-reported hostnames, service banners, an attacker can embed instructions that manipulate the model's output. Greshake et al. demonstrate this against several deployed LLM-integrated applications, including cases where injected instructions cause the model to suppress or distort its report to the user. In a network monitoring context the threat is direct: a compromised device could craft hostnames or service banners that cause the triage LLM to downplay the anomaly it is investigating; more importantly, since PeerWatch runs with elevated privileges prompt injection could even compromise a machine.

PeerWatch addresses this with an architectural boundary described in Section 1.2: the LLM receives only structured summaries from the detection pipeline: numeric scores, event type labels, device identifiers, never raw packet payloads or device-supplied strings. The model operates in an explanation role, producing a structured investigation report; it plays no part in the detection or scoring decision. This design adds interpretability without introducing a new attack surface.

2.4 Tools and Libraries

This section introduces non-obvious tool choices and the rationale behind each. Per the scope stated in Section 1.3, PeerWatch targets hardware a SOHO administrator already owns; tool selection reflects that constraint throughout.

2.4.1 Active Scanning: nmap

Nmap [17] is the active scanning engine for all device discovery and fingerprinting. The choice warrants justification against two faster alternatives.

Masscan [12] is capable of scanning the entire IPv4 address space in under six minutes but performs only port discovery: it carries no OS fingerprint database, no service version detection, and produces no OS confidence scores. On a /24 subnet scanned every five minutes, scan speed is irrelevant; fingerprint depth is not. ZMap [11] is an internet-scale single-packet scanner with the same limitation. Both tools are optimised for breadth; nmap is optimised for depth, which is what per-device identity tracking requires.

Nmap's value in this context is threefold. Its OS detection engine matches response profiles against a database of approximately 5,000 fingerprints, returning both a matched OS family and a confidence score that PeerWatch records alongside the result. Service version detection via banner-grabbing identifies the application behind each open port, enabling service-type drift to be detected independently of port number. Structured XML output carries host metadata, port state, service attributes, and OS match candidates in a single machine-readable document, which PeerWatch's parser (`src/peerwatch/parser.py`) consumes directly rather than relying on a thin wrapper library: full control over confidence fields and timing data is preserved.

2.4.2 Passive Capture: Scapy

Passive fingerprinting requires live packet capture with per-field access to ARP, IP, and TCP headers. Four Python libraries were evaluated.

Raw libpcap via ctypes. Libpcap [15] is the standard C library for low-level packet capture on POSIX systems; it provides a file descriptor over a network interface in promiscuous mode and a BPF (Berkeley Packet Filter) compilation interface for pushing kernel-level packet filters to the capture path. BPF filters are boolean expressions over packet fields (i.e. `arp or (tcp and not port 22)`) that are compiled to a bytecode program executed in the kernel before any packet reaches userspace, reducing the volume of data Python must handle. Accessing libpcap from Python requires ctypes, the standard library module for calling C shared libraries via foreign function interface without writing a C extension. The combination provides maximum throughput but demands significant boilerplate: manual struct layout, pointer arithmetic, and per-protocol field extraction. Appropriate for a production high-throughput capture engine, but not for a research implementation where correctness and readability matter more than throughput on a LAN at the scale we are targeting.

Pyshark. Pyshark [23] wraps tshark [31] — the command-line version of Wireshark, a widely used protocol analyser — as a subprocess, feeding it packets and deserialising its JSON output per packet. The subprocess boundary introduces external process overhead and serialisation cost on every packet; suitable for offline PCAP (packet capture file) analysis, not for real-time per-packet processing in a daemon loop.

Dpkt. Dpkt [29] is efficient for offline PCAP file parsing with a clean Python API, but its live capture support is limited, and it is less expressive than scapy for protocol composition or custom layer access patterns.

Scapy. Scapy [3] is a Python-native packet library where each protocol layer is a first-class Python object, and field access is direct (`pkt[ARP].psrc`, `pkt[IP].ttl`, `pkt[TCP].window`) exactly what passive fingerprinting requires. It supports BPF filter compilation at the capture level, limiting what reaches Python-level handling. Scapy is established in security research with broad protocol coverage and active maintenance. The expressiveness advantage over alternatives outweighs the modest throughput overhead at LAN traffic volumes.

2.4.3 LLM Integration: Ollama, Phi-4, and LangChain

Three related choices form the LLM integration layer.

Ollama [21] is a local inference server that runs quantised open-weights models on commodity hardware via a REST API compatible with the OpenAI client interface. It was selected over direct cloud API providers (OpenAI, Anthropic) because network monitoring context, even in summarised form, must not leave the monitored subnet. An air-gapped deployment must remain functional with no outbound connections; Ollama satisfies this constraint without code changes.

Phi-4 (Microsoft, 14B parameters) is the default model. At 4-bit quantisation it requires approximately 8 GB of RAM, within reach of a dedicated monitoring device the administrator already owns. It was selected over Llama 3 8B (lower resource requirement but weaker structured JSON output reliability observed during development) and Mistral 7B (similar tradeoff) on the basis of structured output quality and instruction-following consistency, both critical for producing well-formed `InvestigationReport` JSON on every invocation. The MIT licence is compatible with the project's open-source goals. The model is configurable; any Ollama-supported model can be substituted via `config.json`.

LangChain [5] provides the `format="json"` grammar-constrained generation interface, ensuring the model produces valid JSON regardless of content, enabling direct deserialisation into a typed Pydantic schema. The alternative of raw HTTP to Ollama's `/api/generate` endpoint was rejected because LangChain's structured output handling, retry logic, and streaming/non-streaming abstraction are non-trivial to reimplement correctly; the investigation report schema is validated against a Pydantic model on deserialisation, and the integration between LangChain's structured output and Pydantic's validation is the key value. If Ollama is unavailable, `SuspiciousAgent` falls back to rule-based severity assignment without invoking LangChain, the dependency is isolated to the LLM path.

2.4.4 Configuration and Validation: Pydantic

All scoring weights, thresholds, and daemon settings are declared in a single Pydantic `PeerWatchConfig` model [8]. Pydantic was chosen over plain dicts or dataclasses for three reasons: automatic type coercion (a JSON string `"3.0"` is coerced to `float` transparently), field-level validation with informative error messages on misconfiguration (a user who sets `suspicion_threshold: "high"` receives an immediate type error rather than a runtime crash deep in the scoring engine), and self-documenting field types and defaults.

The `config.example.json` pattern documents all fields with their defaults; users copy it to `config.json` and override only what they need. `model_validate` handles partial configs, unset fields use the declared defaults. Every component that consumes a threshold receives it from the same config object, eliminating scattered magic numbers and the risk of drift between independently maintained constants.

2.5 Evidence Accumulation and Anomaly Scoring

The signals described in Section 2.2 are individually noisy: a single TTL deviation is consistent with a transient routing change; a single OS fingerprint shift is consistent with a firmware update. Turning noisy per-signal observations into reliable detection requires a principled accumulation model. This section establishes the theoretical basis; Chapter 4 describes the implementation.

2.5.1 Weighted Scoring

Not all signals carry equal evidential weight. A changed SSH host key is near-certain evidence of server substitution; a Jaccard port profile drift below 0.6 is consistent with routine service changes. Assigning uniform weight to all signals would cause low-evidence events to dominate the score when they fire frequently, drowning higher-confidence signals in noise.

Axelsson [2] demonstrates that in intrusion detection systems the false positive rate of individual signals has an outsized effect on overall system utility: even at high true positive rates, a signal with a 1% false positive rate will produce more false alerts than true ones on a typical network where attacks are rare. The practical implication is that signal weights should reflect each signal's discriminative power: its ability to distinguish attack from benign change in the target deployment. PeerWatch assigns weights by evidential strength: cryptographic anchor changes (+3.0 for port/protocol mismatch, +3.0 for SSH host key change) carry far more weight than heuristic signals (+0.5 for Jaccard drift, +0.5 for MAC conflict) that fire routinely under legitimate operation.

2.5.2 Signal Compositing

Independent signals with low individual discriminative power can be combined to achieve high discriminative power. If two signals each have a 10% false positive rate and fire independently, the joint probability of both firing simultaneously on a benign event is approximately 1%, a tenfold reduction in false positive rate with no loss of true positive rate for attacks that trigger both.

This compositing effect is the formal justification for PeerWatch's threshold design: an OS fingerprint drift alone may not cross the investigation threshold; OS drift combined with TTL deviation and port profile change in the same tick reaches the threshold while remaining unlikely under benign conditions. Treating signals as independent is a conservative approximation, in reality, correlated signals (e.g., OS change and TTL change both caused by device substitution) provide less additional evidence than independent signals would, so the approach underestimates joint evidence rather than overstating it.

2.5.3 Exponential Decay and the Cold-Start Period

A device that triggers anomaly events during a legitimate firmware update should not remain permanently penalised once the update is complete and the new baseline is stable. Exponential decay, where the suspicion score halves every T days, implements a forgetting mechanism: unreinforced anomalies fade over time without a hard reset. Event-window approaches that reset scores after a fixed period produce cliff effects (a device near the window boundary retains full score; one just past it retains none); continuous decay avoids this discontinuity. The half-life parameter (`suspicion_half_life_days`, default 3.5 days) is tunable to the deployment context.

The cold-start problem, first observed in recommender systems and fraud detection, applies directly to device identity tracking. A device seen for the first time has no baseline; scoring drift against a single observation produces high false positive rates on legitimate newly-discovered devices. PeerWatch withholds scoring for the first `baseline_min_scans` observations (default 5), recording events without contributing to the suspicion score until a stable baseline is established.

2.5.4 Threshold Calibration

The investigation threshold determines the operating point on the precision-recall curve. Lowering the threshold increases true positive rate at the cost of false positive rate; raising it reduces false alerts at the risk of missed detections. Axelsson [2] shows that for realistic base rates of attacks on a typical network, even modest false positive rates render a detection system impractical; an alert that fires ten times per day on a clean network will be ignored within a week, eliminating any security value.

Threshold calibration is therefore not a parameter to set once and forget but a deployment-specific tradeoff between operator tolerance for false alarms and the acceptable miss rate for real attacks. Chapter 5 characterises PeerWatch's operating point empirically under both simulated attack traffic and clean-traffic conditions, providing the data an operator needs to adjust the threshold for their specific environment.

This chapter has established the foundations on which PeerWatch is built. Section 2.1 defined the threat landscape — the attack techniques that existing single-signal tools cannot detect and that motivate a multi-signal approach. Section 2.2 surveyed the fingerprinting signals available to a locally-deployed monitor, from active nmap probing to passive TCP stack observation and cryptographic anchors. Section 2.3 positioned PeerWatch against existing tools and research, identifying the vacant design space it occupies. Sections 2.4 and 2.5 introduced the tools the implementation relies on and the theoretical basis for combining their outputs into a coherent scoring model. Chapter 3 translates this context into a structured requirements specification and analyses it into an initial design.

3 REQUIREMENTS AND ANALYSIS

3.1 Detailed Problem Statement

The introduction established that existing tools fail to detect device substitution and service-level impersonation attacks on home and small-office networks. This section states the precise technical problem an implementation must solve: the specific uncertainties, constraints, and failure modes from which the requirements in Section 3.2 are derived.

The system must detect when a device observed at a given MAC or IP address in a previous scan is no longer the same physical or logical device. It must do this without external ground truth: at runtime there is no oracle to confirm whether a fingerprint change represents a firmware update or an attack. Detection is therefore probabilistic, evidence accumulates across scan cycles until a configurable confidence threshold is crossed, at which point investigation is triggered. The goal is not to prove an attack has occurred, but to surface sufficient evidence that an operator or autonomous rule can act on it with an acceptable false positive rate. This distinguishes the problem from signature-based intrusion detection, which matches observations against known attack patterns; PeerWatch detects drift against a self-built per-device baseline regardless of whether the underlying technique appears in any signature database.

Several classes of legitimate event produce observations indistinguishable from attacks in isolation. A firmware update changes OS fingerprint and may change port profile and TTL baseline. A DHCP lease renewal reassigns an IP, potentially to a different device. A service restart or version upgrade changes the service banner and may open or close ports transiently. MAC address randomisation, enabled by default in modern mobile operating systems, breaks MAC-keyed identity continuity for mobile devices. A router reconfiguration shifts TTL values and route paths simultaneously for all subnet devices. These events define the false positive ceiling: any signal that fires on them without corroboration or weighting will produce an alert rate that renders the system unusable in practice [2]. Evidence accumulation, signal weighting, and score decay are therefore requirements rather than optional refinements, they are the mechanism by which the system remains useful under legitimate network activity.

The deployment target imposes hard operational constraints. Nmap OS detection requires root privilege to send raw socket probes; promiscuous-mode packet capture requires root and a network interface that can see subnet broadcast traffic. Neither constraint can be mitigated in software. Nmap scanning a /24 subnet takes between 15 and 60 seconds depending on host density and scan depth; the minimum tick interval is bounded below by this figure. Passive capture is continuous while active scans are periodic: passive observations must be attributed to the device identity established by the most recent active scan, and devices that generate no outgoing traffic between scans are invisible to passive observation. There is no managed switching infrastructure, SNMP feed (Simple Network Management Protocol — device telemetry polling), NetFlow export (per-flow traffic metadata generated by managed routers and switches), or SPAN port (Switched Port Analyser — a switch feature that mirrors all traffic from one port to a dedicated monitoring port) available; the monitoring host sees only traffic visible on its local interface and subnet broadcasts.

Per-device threshold analysis has a structural failure mode: a coordinated attack affecting multiple devices simultaneously, each producing insufficient evidence to cross the per-device threshold individually, will produce no alert. Lowering the per-device threshold to catch such attacks would generate unacceptable false positives under legitimate fleet-wide events, a router reconfiguration shifts TTL values for all devices at once. Fleet-level correlation is therefore a first-class detection requirement rather than a sensitivity adjustment. It requires a bounded tick window within which events from multiple peers are co-evaluated; the window start is the previous tick timestamp, and only events recorded after that timestamp are eligible for fleet pattern matching.

Two architectural constraints emerge from the problem domain before any design decisions are made. First, the LLM must not receive adversarially-controlled data. Device-reported hostnames, service banners, and packet payloads are attacker- controllable strings that, if passed to the model, expose it to indirect prompt injection [13], a compromised device could craft its hostname to cause the LLM to suppress or misclassify the alert investigating it. LLM input must therefore be restricted to data derived entirely by the detection pipeline: numeric scores, event type labels drawn from a fixed enum, and device identifiers assigned by PeerWatch, not reported by the device. Second, autonomous blocking carries irreversibility risk: an incorrectly blocked device may take a legitimate service offline. The decision to block must be governed by deterministic rule-based guards that LLM

output cannot override; the model's severity classification may be one guard among several, but the guard logic itself must be purely rule-based.

3.2 Requirements

The requirements below are derived directly from the constraints identified in Section 3.1. Functional requirements specify what the system must do; non-functional requirements specify properties the implementation must exhibit. Requirements are numbered for traceability: Chapter 4 references them when describing design decisions, and Chapter 5 references them when describing test coverage.

3.2.1 Functional Requirements

ID	Requirement
A — Device Discovery and Identity Tracking	
FR1	The system shall periodically scan a configurable IPv4 /24 subnet using nmap, discovering active hosts and recording OS family, open TCP/UDP ports, service types, and MAC address per host.
FR2	Device identity shall be persisted across daemon restarts in a structured store keyed by MAC address; devices that expose no MAC address shall be keyed by IP address.
FR3	On each scan, the system shall compare each device's current fingerprint against its stored baseline and emit a typed identity event for each detected drift.
FR4	Each identity event shall carry a configurable numeric weight; the device's suspicion score shall be the weighted sum of accumulated events subject to exponential decay between ticks.
FR5	The system shall maintain a per-device known-services table; events for service fingerprint changes on ports where oscillation has been previously observed shall be suppressed.
FR6	Scoring shall be withheld for the first N observations of a newly discovered device (configurable `baseline_min_scans`); events shall be recorded but shall not contribute to the suspicion score during this warmup period.
B — Passive Monitoring	
FR7	The system shall run a continuous passive packet capture loop in parallel with active scans, monitoring: (a) ARP reply binding conflicts; (b) per-device TTL deviation from an accumulated baseline; (c) TCP stack fingerprint inconsistency; (d) IP identification field counter anomalies.
FR8	The system shall track route path stability per device via periodic traceroute; each hop shall be attributed to an autonomous system; changes in hop sequence or ASN membership shall emit identity events.
FR9	The system shall track SSH host key and TLS certificate fingerprints per service port; a change in fingerprint on a previously observed port shall emit an identity event.
C — Fleet Correlation	
FR10	After each scan tick, the system shall evaluate identity events recorded since the previous tick across all peers; where $\geq N$ peers exhibit the same event class (pattern-dependent

threshold), a fleet pattern shall fire and a configurable score boost shall be applied to each matching peer.

D — LLM-Assisted Triage

FR11 When a device's suspicion score reaches or exceeds a configurable threshold, the system shall invoke a locally-running LLM via Ollama and request a structured JSON investigation report.

FR12 The system shall remain fully operational if Ollama is unavailable; a rule-based severity assignment shall be substituted for the LLM report without loss of detection capability.

FR13 The LLM prompt shall contain only data derived by the detection pipeline: numeric suspicion score, event type labels from a fixed enum, system-assigned device identifiers, and timestamp. Raw packet payloads, device-supplied hostnames, service banners, and any other attacker-reachable strings shall not appear in the prompt.

E — Remediation

FR14 The system shall support autonomous remediation via iptables INPUT and OUTPUT DROP rules for devices that satisfy all blocking guards.

FR15 All four of the following guards must pass before a block is applied: (a) IP and MAC not in `never_block`; (b) suspicion score $\geq$ `block_confidence_floor`; (c) severity == "high"; (d) no active block already exists for this IP.

FR16 Three remediation modes shall be supported: `dry_run` (log decision only), `confirm` (prompt operator before applying), `enforce` (apply rule immediately).

FR17 Blocks shall expire after a configurable TTL; expired rules shall be removed on each tick. All block decisions including dry-run outcomes shall be written to an append-only audit log before any rule is applied or withheld.

F — Configuration

FR18 All thresholds, event weights, decay parameters, scan interval, remediation mode, and model selection shall be configurable via a single JSON file without code changes.

Table 1: Functional requirements.

3.2.2 Non-Functional Requirements

ID	Requirement	Category
NFR1	The system shall run on hardware with $\geq$ 4 GB RAM with no dedicated appliance or managed switching infrastructure required.	Deployment
NFR2	No outbound internet connectivity shall be required at runtime; the system shall be fully functional in an air-gapped environment.	Deployment
NFR3	The LLM prompt construction path shall contain no dynamic interpolation of device-supplied strings, satisfying FR13 by construction rather than by policy.	Security
NFR4	All remediation decisions shall be written to the audit log before any iptables rule is applied or withheld; no silent blocking shall occur.	Auditability
NFR5	The scan interval shall be bounded below by a configurable rate-limit floor to prevent subnet flooding regardless of operator configuration.	Performance

NFR6	The detection and scoring pipeline shall operate independently of LLM availability, satisfying FR12 without degraded detection coverage.	Reliability
NFR7	The test suite shall require no live network access; all detection scenarios shall be exercisable via simulated scan sequences injected directly into PeerStore.	Testability
NFR8	The system shall build device baselines autonomously from a cold start with no operator-supplied initial configuration beyond the target subnet.	Usability

Table 2: Non-functional requirements.

3.3 Use Cases

Four actors interact with the system. The **Network Administrator** is the human operator: they configure PeerWatch, review investigation reports, approve or reject remediation actions in `confirm` mode, and consult the audit log. The **PeerWatch Daemon** is the primary automated actor: it drives the scan loop, ingests passive observations, performs fingerprint comparison, and coordinates all downstream components. **Ollama** is an external actor: a local LLM service invoked by the daemon when suspicion crosses the investigation threshold; it is never invoked directly by the administrator. **iptables** is a system actor that receives firewall rule commands from the Remediator component.

Use case diagram (see source: docs/usecase.puml)

Figure 1: PeerWatch use case diagram.

Table 3 lists all ten identified use cases. Full specifications for the three most security-critical cases (UC4, UC3, UC7) appear in Appendix B.

ID	Use Case	Primary Actor
UC1	Run periodic subnet scan	Daemon
UC2	Ingest passive observations	Daemon
UC3	Compare fingerprint against baseline	Daemon
UC4	Trigger LLM investigation	Daemon

ID	Use Case	Primary Actor
UC5	View investigation report	Administrator
UC6	Configure parameters	Administrator
UC7	Approve remediation block	Administrator
UC8	Review audit log	Administrator
UC9	Inject crafted nmap XML	Administrator
UC10	Auto-expire block on TTL	Daemon

Table 3: Identified use cases.

UC4 (*Trigger LLM investigation*) is architecturally the most constrained use case. The daemon passes only sanitised, structured fields (suspicion score, event list, and fingerprint delta) to the agent prompt; raw packet payloads and peer-supplied strings are never forwarded. This design directly mitigates the indirect prompt-injection risk identified in NFR3 [13]. UC7 (*Approve remediation block*) is exercised only in `confirm` mode; in `dry_run` mode it reduces to a no-op, and in `enforce` mode it is bypassed entirely by the daemon. UC9 (*Inject crafted nmap XML*) is included as an explicit use case because the injection interface (dropping a file into `data/raw/` between ticks) forms the basis of both the attack simulation test suite and the manual demonstration scenario described in Chapter 5.

3.4 Analysis

The requirements established what the system must do; this section analyses them to extract the information structures the system must store, the components that must exist to satisfy each requirement group, and the key architectural decisions that constrain the design before implementation begins. These decisions are elaborated further in Chapter 4.

3.4.1 Data Model

The central entity is a **Peer**: a device that has been observed on the subnet in at least one scan. A Peer record accumulates identity evidence across scan cycles: it is not a snapshot of a single observation but a longitudinal model of a device's expected behaviour.

Stable devices (those whose nmap output includes a MAC address) are keyed by MAC address, since IP addresses are volatile: a DHCP lease renewal or router reboot can legitimately reassign an IP to a different device, which would cause a MAC-keyed system to raise a false positive only for the new device, not the old one. Devices that nmap reports without a MAC (typically off-subnet hosts or devices that suppress ARP) are keyed by IP address and flagged as volatile; identity continuity guarantees for these peers are weaker by design.

Table 4 summarises the fields maintained per Peer and the requirement group (Table 1) each field satisfies.

Field	Description	FR group
`mac`	Primary key for stable devices; `None` for volatile peers	A
`ip`	Last observed IP address; primary key for volatile peers	A
`os_family`	Operating system family inferred by nmap	B

Field	Description	FR group
`open_ports`	Set of open TCP/UDP ports from most recent scan	B
`known_services`	Per-port service type baseline (suppresses oscillation)	B
`ttl_baseline`	Mean TTL observed in passive capture; used for deviation scoring	C
`tcp_fingerprint`	Passive OS fingerprint from TCP stack behaviour (p0f model)	C
`route_path`	Ordered hop sequence from last traceroute	C
`ssh_key_hash`	SHA-256 of SSH host key on port 22 (or configured port)	B
`tls_cert_hash`	SHA-256 of TLS certificate on known TLS ports	B
`mac_oui_vendor`	OUI vendor string derived from MAC prefix	B
`suspicion_score`	Accumulated weighted evidence score; decays over time	D, E
`baseline_scans`	Count of scans completed; gates warmup period	A
`scan_history`	Ordered list of `ScanRecord` snapshots	A, B

Table 4: Peer entity fields and the requirement groups they satisfy.

Three supporting entities complete the model. A **ScanRecord** is an immutable snapshot of one tick: it stores the timestamp, the list of anomaly events fired, and the score delta applied. A **FleetAlert** records a coordinated-pattern detection: the pattern name, the set of peer identifiers involved, the boost applied, and the tick timestamp. A **RemediationBlock** is the audit record for one iptables rule: the target IP and MAC, the issue time, the expiry time, the triggering severity, and the score at the time of issue. An **InvestigationReport** stores the structured JSON output of one LLM invocation: the peer identifier, severity verdict, rationale text, and recommended action. None of these supporting entities reference each other; they are all associated to their parent Peer by MAC or IP key, keeping the model flat and easy to serialise to JSON.

3.4.2 Component Identification

Table 5 maps each functional requirement group to the source module responsible for satisfying it. The mapping guided the initial module decomposition and is reflected directly in the `src/peerwatch/` package structure.

FR group	Module(s)	Responsibility
A	`daemon.py`, `parser.py`	Periodic nmap invocation, XML ingestion, tick coordination
B	`peer_store.py`, `comparator.py`	Device identity storage, fingerprint comparison, drift scoring
C	`packet_capture.py`, `route_tracker.py`	Passive TTL/TCP observation, traceroute path tracking
D	`fleet_correlator.py`	Cross-peer coordinated pattern detection, score boosting
E	`agent.py`	LLM triage invocation, rule-based fallback severity assignment
F	`remediation.py`	iptables rule management, block TTL, audit logging

FR group	Module(s)	Responsibility
—	`config.py`	Pydantic configuration model, threshold and weight validation

Table 5: Functional requirement groups mapped to implementation modules.

The daemon coordinates all modules but contains no detection logic itself; it is responsible only for scheduling, calling `convert_pending_xml()` to ingest any injected nmap files, and advancing the tick. This separation means each detection component can be unit-tested in isolation without running the full scan loop, which is important given that live nmap scans require root and network access.

3.4.3 Key Design Decisions

Three decisions made during analysis have the broadest impact on the rest of the design. Each was reached by considering alternatives against the requirements and constraints established above.

MAC-primary keying. The alternative was to key peers by IP address. IP-keyed identity breaks immediately on DHCP lease renewal: if `192.168.1.42` is reassigned from a printer to a laptop, an IP-keyed store would score the laptop for not looking like the printer. A hostname-keyed approach was also considered, but hostname resolution requires DNS, which is not always present or reliable on small-office networks, and hostnames can be spoofed as readily as IP addresses. MAC addresses are hardware-assigned and stable across reboots, making them the strongest available identity anchor at layer 2. The MAC-spoofing case, where an attacker clones a victim's MAC, is explicitly detected as an identity collision event (FR8) rather than suppressed.

Additive score accumulation with exponential decay. An event-threshold model, fire an investigation on the first tick any single event exceeds a fixed magnitude, would satisfy FR13 at low implementation cost, but would violate NFR1 (false-positive rate). Axelsson's analysis of base-rate effects in intrusion detection [2] shows that even a 99% accurate binary classifier produces predominantly false positives on a network where attacks are rare; accumulating evidence across multiple ticks raises the posterior probability of a genuine attack before triggering investigation. Exponential decay (score halves every 3.5 days by default) prevents stale low-confidence events from permanently biasing the score of a peer that was suspicious six months ago and has been well-behaved since. The cold-start warmup period (first five scans, configurable) records events without scoring them, establishing a baseline before the decay clock starts.

LLM as triage advisor, not decision-maker. An alternative design placed the LLM in the authorisation path: the agent would both assess severity and issue the iptables command. This was rejected on two grounds. First, auditability: FR18 requires a complete audit trail; LLM reasoning is non-deterministic and cannot be reproduced from the audit record alone. Second, reliability: if Ollama is unavailable, the system must still function; rule-based severity assignment serves as the fallback, and the remediation guards are purely deterministic. The adopted design keeps the LLM solely in the investigation path: it produces a structured JSON report that a human or the rule-based guards can act on, but it never issues a system call.

Chapter 4 describes the implementation of each component in detail, starting with the daemon scan loop and progressing through the detection pipeline in the order data flows through it.

4 DESIGN AND IMPLEMENTATION

4.1 System Architecture

PeerWatch is a single daemon process (`daemon.py`) executing a periodic tick loop. Each tick is an atomic, sequential pipeline: it scans the subnet, ingests results into the device identity store, drains the passive observation queue, runs fleet-level correlation, triggers LLM investigation for peers whose suspicion score crosses the threshold, and applies or expires remediation blocks. The sequential tick model eliminates inter-component race conditions: no component reads state that another is concurrently writing, and the full system state is reproducible from the persisted `peer_store.json` at any tick boundary.

Figure 1 shows the data flow between components. Solid-bordered boxes are Python modules; dashed-bordered boxes are external processes or files. Italicised labels denote components outside the Python process boundary. `PeerStore` (shaded) is the central mutable state shared by all detection phases.

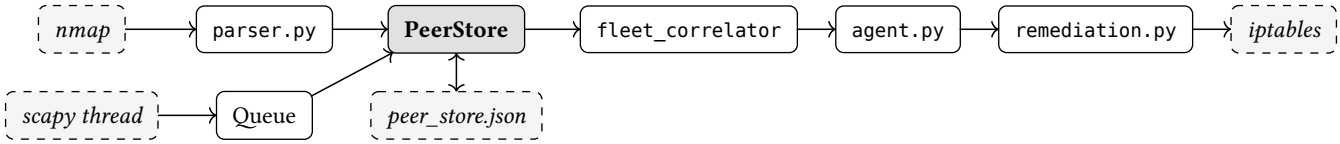

Figure 1: PeerWatch component data-flow diagram. Dashed boxes are external to the Python process. `PeerStore` is the central identity state updated by both ingestion paths.

Tick pipeline. Each tick executes the following steps in order:

1. **Rate-limit check.** If the previous tick completed fewer than `min_scan_interval_minutes` ago, the tick is skipped to prevent scan storms during slow nmap runs.
2. **XML injection drain.** `convert_pending_xml()` processes any crafted nmap XML files dropped into `data/raw/` since the last tick and writes the converted JSON to `data/processed/`. This is the test injection interface (UC9).
3. **nmap scan.** The daemon invokes nmap as a subprocess with `-O -sV -oX`; the resulting XML is parsed by `parser.py` into a list of `NormalisedData` records.
4. **Active ingest.** `add_or_update_peer()` is called once per host returned by nmap, comparing each against its stored fingerprint and accumulating anomaly events and score deltas.
5. **Passive drain.** Observations accumulated by the background capture thread since the last tick are drained from the inter-thread queue and applied to peer records via the corresponding `PeerStore` ingestion methods.
6. **Fleet correlation.** `FleetCorrelator.analyse()` examines events across all peers and applies coordinated-pattern score boosts.
7. **Investigation.** For each peer whose `suspicion_score` meets or exceeds `suspicion_threshold`, `SuspiciousAgent.investigate()` is called.

8. **Remediation.** `Remediator.evaluate()` checks guard conditions for each peer and `act()` issues or skips iptables blocks; `unblock_expired()` removes blocks whose TTL has elapsed.
9. **Persist.** `peer_store.save()` serialises the full store to `peer_store.json`.

Thread model. The passive capture layer (`packet_capture.py`) is implemented as a self-contained background component and is fully functional as a library; however, it is not yet started by `daemon.py` in the current release — the daemon tick runs only the active-scan pipeline (steps 1–4 and 6–9). The architecture below describes the intended integration, which is the natural next step for deployment. When wired in, passive packet capture cannot share the main thread because scapy's `sniff()` is a blocking call: a single `threading.Thread(daemon=True)` would be started once at daemon startup (before the first tick), running the capture loop indefinitely. (`daemon=True` means the thread exits automatically when the main process exits, with no explicit join required.) Inter-thread communication uses a `queue.Queue`: the capture thread enqueues observation records and the main thread drains the queue at step 5 of each tick. All `PeerStore` ingestion methods acquire a `threading.Lock` (§4.3), so passive observations drained on the main thread and any future concurrent callers are safe.

Persistence model. `PeerStore` serialises to `peer_store.json` via Pydantic's `model_dump()` at the end of each tick. The file is loaded at daemon startup if it exists, allowing the daemon to resume from prior state after a restart without discarding accumulated suspicion evidence. `PeerStore.last_tick_at` is persisted alongside the peer data; `FleetCorrelator` uses it as the event-window start to avoid re-triggering fleet patterns from events that fired in a previous session.

Configuration injection. All components receive a single `Config` instance constructed at startup. No module reads configuration from a global variable; each takes `config` as a constructor argument. This makes every component independently testable with a minimal config object, without touching the filesystem or invoking nmap.

The following sections describe the design and implementation of each component in the order data flows through the pipeline.

4.2 Scan Loop and Data Ingestion

`daemon.py` is the entry point and tick coordinator. It contains no detection logic: its responsibilities are scheduling, nmap subprocess management, file I/O, and dispatching each tick's results to the pipeline modules. This separation means the entire detection pipeline can be exercised in tests by calling `run_pipeline()` directly with pre-supplied JSON files, without invoking nmap or touching the network.

4.2.1 nmap Invocation

Each tick invokes nmap as a subprocess:

```
subprocess.run(
    ["nmap", "-sV", "-O", "--osscan-guess", "-oX", str(output_path), subnet],
    capture_output=True, text=True, timeout=300,
)
```

Three flags drive the identity model. `-sV` enables service and version detection, populating the per-port service strings that PeerStore uses for service-drift comparison. `-O` enables OS detection, which requires root; if the daemon runs without elevated privileges, `os_candidates` will be empty for every host and OS-drift events will never fire. `--osscan-guess` instructs nmap to emit all plausible OS candidates, not only its top pick — this is what makes the `os_candidates` multi-entry dictionary possible and is therefore essential to the false-positive suppression described in §4.3.

A 300-second timeout guards against unresponsive hosts stalling the scan loop. Non-zero nmap exit codes are logged but do not abort the daemon: a failed scan produces no new JSON file, so the pipeline runs on the previous tick's data with no new ingestion, and the loop continues.

4.2.2 XML to JSON Conversion and Field Stripping

`jsonify_xml()` parses nmap's XML output using `xmltodict` — a lightweight Python library that maps XML element hierarchies to Python dicts — and writes a cleaned JSON file to `data/processed/`. Before writing, a set of volatile fields is removed from each host record:

```
_STRIP_FIELDS = {
    "@starttime", "@endtime", "distance", "tcpsequence",
    "ipidsequence", "tcptssequence", "times", "hostnames",
}
```

These fields change on every scan (TCP ISN sequencing counters, scan timestamps, and round-trip time measurements) but carry no persistent identity signal. Retaining them would inject noise into fingerprint comparisons and inflate the stored JSON with data that cannot distinguish a firmware update from an impersonation. Stripping happens at conversion time, so the fields are absent from both the on-disk JSON and the structures ingested by PeerStore.

4.2.3 The Injection Interface

`convert_pending_xml()` polls `data/raw/` for XML files that do not yet have a JSON counterpart in `data/processed/`:

```
def convert_pending_xml(output_dir: Path) -> None:
    for xml_path in RAW_DIR.glob("*.xml"):
        json_equiv = output_dir / (xml_path.stem + ".json")
        if not json_equiv.exists():
            logging.info(f"New XML detected: {xml_path.name}")
            jsonify_xml(xml_path, output_dir)
```

Any XML file dropped into `data/raw/` between ticks is converted and ingested in the next tick, without requiring root access or a live network. This interface is the basis of the entire attack simulation test suite described in Chapter 5: crafted nmap XML files representing spoofed devices, ARP poisoning scenarios, and service impersonation attacks can be injected and processed under controlled conditions. The injection check runs before the live nmap scan each tick, ensuring manually supplied files are processed in the same tick they appear.

4.2.4 NormalisedData and the os_candidates Design

`NmapParser` transforms one xmltodict host record into a `NormalisedData` Pydantic model. Most fields are straightforward extractions; the `os_candidates` field warrants attention:

```
candidates: dict[str, int] = {}
for match in osmatches:
    accuracy = int(match.get("@accuracy", 0))
    for osclass in match.get("osclass", []):
        family = osclass.get("@vendor") or osclass.get("@osfamily")
        if family:
            candidates[family] = max(candidates.get(family, 0), accuracy)
```

nmap returns multiple `osmatch` entries ordered by descending accuracy. Rather than recording only the top-ranked family, `NmapParser` iterates all entries and builds a dict mapping each OS family to its best accuracy score across all matches i.e. `{"Linux": 96, "Google": 93}` for an Android device. This representation matters because nmap's top pick is not deterministic: slight variation in TCP response timing between scans can cause it to reorder "Linux" and "Microsoft Windows" as the top result for the same device. If PeerStore compared only the top candidate, this reordering would fire an OS-change event on every other scan. By comparing candidate sets (§4.3), PeerStore fires only when the union of known candidates and newly observed candidates diverges, which is robust to nmap's non-determinism.

Port extraction handles two xmltodict edge cases: when nmap reports exactly one open port, xmltodict returns a dict rather than a list, so `_extract_ports()` normalises both forms. Service strings are constructed as `name + "-" + product` where both are present (e.g. `ssh-OpenSSH`), falling back to whichever is available. `open_ports` is sorted before storage so that Jaccard distance computation in PeerStore is independent of scan ordering.

4.2.5 Ingestion Tracking, Rate Limiting, and Shutdown

`PeerStore.ingested_scan_files` is a set of JSON filenames that have already been processed. `run_pipeline()` filters `data/processed/*.json` against this set before ingesting, ensuring that daemon restarts do not replay historical scans and that each injected file is ingested exactly once. The set is persisted with `peer_store.json` at the end of each tick.

The rate-limit guard compares the elapsed time since `last_scan_at` against `min_scan_interval_minutes` (default 2 minutes). If a slow scan overlaps its scheduled successor, the next tick is deferred rather than stacked, preventing two nmap processes from running concurrently.

Graceful shutdown is handled by registering SIGINT and SIGTERM handlers that set a `shutdown` boolean flag. The inter-tick sleep is implemented as `_sleep_interruptible`, which wakes every second to check the flag. The daemon therefore always completes its current tick, including persisting `peer_store.json`, before exiting — ensuring no partially-applied score increments are lost on SIGTERM.

4.3 Device Identity Store and Fingerprint Comparison

`PeerStore` is the central mutable state of the system. It maintains a longitudinal model of each observed device, accumulates anomaly evidence across scan cycles, and exposes ingestion methods for all five detection signals. All public methods that mutate peer state acquire a `threading.Lock`, since the passive capture thread calls ingestion methods concurrently with the main thread's active-scan ingestion.

4.3.1 Peer Model and Identity Keying

Each device is represented by a `Peer` record keyed by an internal UUID. Two auxiliary indexes provide fast lookup: `mac_to_id` maps normalised MAC address to UUID, and `ip_to_id` maps IP address to UUID. Stable devices, those whose nmap output includes a MAC address, are registered in both indexes; volatile (MAC-less) devices appear only in `ip_to_id` and carry an `is_volatile` flag.

`add_or_update_peer()` resolves which peer a new scan result belongs to by querying both indexes and branching on the number of candidates found:

- **Zero candidates** — the device is new; `_create_peer()` is called. The first scan's service types are pre-loaded into `known_services` so day-one observations do not immediately trigger service-change events.
- **One candidate** — the existing peer is updated. Suspicion decay is applied first, then fingerprint comparison, then the peer record is refreshed.
- **Multiple candidates** — the MAC index and IP index resolved to different peer records, indicating a possible identity collision; `_resolve_conflict()` is invoked.

4.3.2 Fingerprint Comparison Engine

`_compare_fingerprints()` evaluates four checks in sequence and returns a `FingerprintComparison` record containing the events that fired and an overall similarity score.

OS comparison via candidate-set intersection. Rather than comparing the single top-ranked OS pick, the comparison uses the full set of OS family names across all `osmatch` entries:

```
prev_families = _os_candidate_families(prev)
curr_families = _os_candidate_families(incoming)
os_match = bool(prev_families & curr_families)
if not os_match:
    events.append("os_family_changed")
```

This choice is motivated by nmap's non-determinism: a device at 95% confidence for both "Sony" and "Linux" may flip its top-ranked entry between scans depending on TCP response timing. Set intersection is stable under this jitter: `os_family_changed` fires only when the candidate sets are genuinely disjoint, i.e. no previously observed OS family appears in the new scan at all.

Port-set Jaccard similarity. Open port sets are compared as:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

With A and B being the previous and incoming port sets. A result below `port_jaccard_threshold` (default 0.6) fires `port_profile_changed` (+0.5). Empty unions are excluded from comparison to avoid penalising devices behind a firewall that intermittently suppresses all ports.

Service type comparison on shared ports. Only the first token of nmap's service string is compared: `ssh` from `ssh-OpenSSH`, `http` from `http-nginx`. Version upgrades within the same protocol family are therefore not flagged — only a genuine protocol change on a port (e.g. `ssh`

→ `http`) triggers `service_type_changed` (+1.0 per port). Comparison is limited to `shared_ports = prev_ports & curr_ports`; a port present in only one scan is captured by the Jaccard check, not here.

Full identity shift. A composite event that fires when all three dimensions change simultaneously: OS families disjoint, Jaccard below 0.4, and at least one service type changed or no shared ports at all. This is the primary device-substitution signal (+2.0) and is designed to be robust against the single-signal false positives that motivated the multi-signal approach.

The overall similarity score used for logging and drift reporting (not for triggering events) is:

$$s = 0.5 \cdot s_{\text{OS}} + 0.3 \cdot J + 0.2 \cdot s_{\text{svc}}$$

where $s_{\text{OS}} = \frac{|F_{\text{prev}} \cap F_{\text{curr}}|}{|F_{\text{prev}} \cup F_{\text{curr}}|}$ is the Jaccard similarity of the two OS candidate-family sets (not a binary flag, so partial overlap (e.g. one family in common out of three) gives a score between 0 and 1); J is the port-set Jaccard from the previous check; and s_{svc} is the fraction of shared ports whose service type was unchanged ($s_{\text{svc}} = 1 - \frac{|\text{changed ports}|}{|\text{shared ports}|}$, or 1.0 when there are no shared ports).

4.3.3 False-Positive Suppression

Three distinct mechanisms address recurring false-positive patterns identified during development.

`known_services` oscillation suppression. `Peer.known_services` is a per-port set of service type strings that have been observed at least once. Before recording a `service_type_changed` event, the incoming service type is checked against this set:

```
new_type = new_svc.split("-")[0] if new_svc else ""
if new_type in prev.known_services.get(port, set()):
    # oscillation — silently re-merge and skip
    continue
```

If the type has been seen before, the event is silently skipped and the old type is re-added to the set. Without this guard, a Chromecast whose nmap alternates between `ajp13` and `castv2` on port 8009 across consecutive scans would fire a service-change event every other tick.

Warmup period. For the first `baseline_min_scans` scans (default 5), anomaly events are recorded in `identity_history` but the suspicion score is not incremented. This prevents a device first observed mid-configuration from entering the store with artificially elevated score.

One-shot guards. `flagged_port_mismatches: set[int]` and `flagged_vendor_mismatch: bool` are set when a port-protocol mismatch or MAC OUI/OS mismatch is first detected and prevent those checks from re-scoring on every subsequent tick. Both fields are persisted in the peer snapshot, so the guards survive daemon restarts.

Port-protocol mismatch. `WELL_KNOWN_PORT_PROTOCOLS` maps nine well-known ports to the set of service types legitimately expected on them: port 22 expects `ssh`, port 80 expects `http`, and so on. Any other service type detected on these ports fires `port_protocol_mismatch` (+3.0), the highest

single-event score in the system, reflecting that running a non-standard service on a well-known port is a strong indicator of a backdoor or replacement device. Port 443 deliberately includes both `https` and `http` since nmap may report `http` when TLS termination is handled transparently by a proxy. Service identifications of `tcpwrapped` (where nmap could not determine the protocol) are excluded to avoid false positives on port-forwarded services.

MAC OUI vendor versus OS family. `VENDOR_OS_COMPATIBILITY` maps vendor name substrings to the set of OS families compatible with that hardware. A device with an Apple-registered MAC but a Linux OS fingerprint fires `mac_vendor_os_mismatch` (+2.0). Generic NIC vendors (Intel, Realtek, ASUSTek) are intentionally absent from the table: commodity PC hardware can run any OS, so flagging it would produce a high false positive rate. Only vendors with a strongly constrained hardware ecosystem (Apple, Raspberry Pi, Microsoft Xbox, Sony, Amazon) are included. Matching uses substring search (`keyword in vendor.lower()`) to handle vendor string variations in nmap output.

4.3.4 Scoring and Exponential Decay

Suspicion score is additive across events and ticks. Decay is applied at the start of each `add_or_update_peer()` call, before comparison:

```
peer.suspicion_score *= 0.5 ** (elapsed_days / self._cfg.suspicion_half_life_days)
```

Applying decay *before* scoring means a peer that has been clean for a month arrives at the comparison with near-zero prior evidence; new events are therefore not amplified by stale history. The default half-life is 3.5 days: a score of 3.0 (the investigation threshold) decays below 1.0 in approximately one week of clean scans.

Table 6 summarises the score increments for all active-scan events.

Event	Score
`os_family_changed`	+2.0
`full_identity_shift`	+2.0
`mac_vendor_os_mismatch`	+2.0
`service_type_changed`	+1.0 per port
`identity_conflict_detected`	+1.0
`port_profile_changed`	+0.5
`mac_conflict`	+0.5
`port_protocol_mismatch`	+3.0

Table 6: Active-scan suspicion score increments. Passive-capture and fleet events are covered in §4.4 and §4.5.

4.3.5 Conflict Resolution, Volatile Peers, and Persistence

Identity conflict resolution. When `add_or_update_peer()` resolves multiple candidate IDs (MAC and IP map to different peer records) `_resolve_conflict()` selects a survivor and merges the losers into it. Survivor selection prefers the non-volatile (MAC-confirmed) peer; among equally volatile candidates, the higher scan count wins, favouring the record with more accumulated evidence.

`_merge_peers()` transfers history, IP set, `known_services`, and score from loser to survivor before deleting the loser record. The conflict records `identity_conflict_detected` (+1.0) on the survivor.

This situation arises in two scenarios: a MAC-spoofing attack where the attacker takes a known IP without cloning the MAC (the IP maps to the victim's peer, the attacker's MAC maps to no peer, collision at the IP boundary), and a legitimate DHCP reassignment where an IP previously held by peer A is now claimed by a new device with a different MAC. The system treats both conservatively: score is incremented and investigation is triggered if the threshold is reached, leaving the operator to distinguish the cases.

Volatile peer eviction. MAC-less peers not observed within `volatile_peer_ttl_hours` are removed by `evict_stale_volatile_peers()`, called once per tick after ingestion. This bounds memory growth on subnets with high DHCP churn where off-subnet hosts transiently appear without MAC addresses.

Persistence. `PeerStore.save()` writes a versioned JSON snapshot containing all peer records, `ingested_scan_files`, and `last_tick_at`. `PeerStore.load()` rebuilds the `mac_to_id` and `ip_to_id` indexes from the peer data rather than storing them separately, ensuring index consistency on load. A `_SNAPSHOT_VERSION` integer guards against loading snapshots from an incompatible schema — a mismatch logs a warning and starts a fresh store rather than attempting migration.

Comparator. `Comparator` is a read-only reporter that takes a populated `PeerStore`, aggregates event counts per peer into `PeerDriftSummary` records, sorts them by suspicion score descending, and logs a formatted drift table to the daemon log. It contains no detection logic and makes no mutations; it is called once per tick after ingestion to provide a human-readable audit trail independently of the alert output.

4.4 Passive Observation Layer

The passive observation layer collects network signals continuously between nmap scans, providing four detection channels that active scanning cannot deliver: TTL baseline deviation, ARP spoofing, TCP stack fingerprint contradiction, and IP ID counter anomaly. A fifth channel, route path stability, runs on demand via traceroute.

4.4.1 Architecture and Testability

The layer is split into two halves that share no direct dependency. The first half consists of typed observation dataclasses ,`TTLObservation`, `ARPObservation`, `TCPFingerprintObservation`, and `IPIDObservation`, that are plain Python `@dataclasss` with no scapy import. The second half is the capture machinery: `PassiveCaptureObserver` and `SniffCaptureLoop`.

This separation is the key testability decision for Phase 2. Because the observation types are scapy-free, the full test suite for passive detection injects typed observations directly into `PeerStore.ingest_*()` without requiring a live network interface, elevated privileges, or scapy installed. A test that verifies ARP spoofing detection simply constructs an `ARPObservation` with a conflicting MAC and calls `peer_store.ingest_arp_observation()` — no packet capture involved.

`PassiveCaptureObserver` wires the two halves. It maintains four callback lists and exposes `on_ttl()`, `on_arp()`, `on_tcp_fingerprint()`, and `on_ip_id()` registration methods. `process_packet()` parses a raw scapy packet into the appropriate observation type and dispatches to all registered callbacks. The daemon registers one callback per type that routes observations to the corresponding `PeerStore.ingest_*()` method.

`SniffCaptureLoop` wraps scapy's `sniff()` in a daemon thread:

```
sniff(
    iface=self.iface,
    filter=self.bpf_filter,   # "ip or arp"
    prn=self.observer.process_packet,
    store=False,
    timeout=timeout,
)
```

The BPF filter `"ip or arp"` is applied at kernel level before any Python code runs, discarding non-IP, non-ARP traffic without waking the Python interpreter. `store=False` discards packets after processing, preventing unbounded memory accumulation in long-running sessions. The thread is a daemon thread so it exits automatically when the main process ends, without requiring an explicit stop call.

4.4.2 Per-Signal Capture

TTL baseline and deviation. Real-world TTL values observed in packets are always less than or equal to the device's origin TTL, since each router along the path decrements the field by one. `snap_ttl_to_os_default()` exploits this property to recover the plausible origin:

```
_TTL_THRESHOLDS = [64, 128, 255]   # Linux/macOS, Windows, Cisco

def snap_ttl_to_os_default(raw_ttl: int) -> int:
    for threshold in _TTL_THRESHOLDS:
        if raw_ttl <= threshold:
            return threshold
    return _TTL_THRESHOLDS[-1]
```

An observed TTL of 60 snaps to 64 (four-hop Linux host); 120 snaps to 128 (eight-hop Windows host). PeerStore builds a per-peer TTL baseline from the first `ttl_baseline_min_samples` observations, using the median to resist outlier packets, then snaps to an OS default. Once established, any observation deviating by more than `ttl_deviation_threshold` (default 15) fires `ttl_deviation` (+2.0). A deviation of this magnitude cannot be explained by a route-length change alone — it implies a different OS-family origin TTL or an injected packet with a forged TTL.

ARP spoofing detection. Only ARP reply packets (`op == 2`, is-at) are processed. Each reply's claimed IP-to-MAC binding is checked against the confirmed MAC stored in PeerStore for that IP. A discrepancy fires `arp_spoofing_detected` (+3.0), the highest-weight passive signal. This directly detects the attack described in Section 2.1: an attacker broadcasting gratuitous ARP replies to redirect subnet traffic through a machine they control.

TCP passive fingerprint. TCP SYN packets are fingerprinted passively: SYN-only (`SYN` flag set, `ACK` not set) because the SYN carries the initiating device's native TCP stack options unmodified. A SYN-ACK would reflect the responding server's preferences, not the device being monitored. Three OS profiles are maintained in `TCP_FINGERPRINT_PROFILES`, drawn from p0f and nmap OS templates [32]:

```
"Linux": {
    "option_signatures": [
        ["MSS", "SACK", "TS", "NOP", "WScale"],
        ["MSS", "NOP", "NOP", "TS", "NOP", "WScale", "SACK"],
    ],
    "window_hint": range(14600, 65536),
    "ttl": 64,
},
```

`infer_os_from_tcp_fingerprint()` scores each profile against the observed option-kind sequence: +1.0 per option present in both signature and observation, −0.5 per expected option absent, −0.3 per extra option present; +1.0 bonus for exact window size match, +0.5 for window in range. A minimum score of 2.0 is required to suppress guesses on sparse input. The inferred OS is cross-referenced against the peer's nmap `os_candidates` set via a family mapping (e.g. `"Linux"` covers both `"Linux"` and `"Android"` in nmap output). A contradiction fires `tcp_fingerprint_mismatch` (+2.0).

IP ID counter anomaly. The IP Identification header field is a 16-bit counter. Older operating systems (and Windows) maintain a single global counter that increments monotonically; modern Linux assigns a random value per connection. `_detect_sequential_ip_ids()` distinguishes the two behaviours by examining the median per-step delta across a sample window:

```
deltas = [(samples[i+1] - samples[i]) % 65536 for i in range(len(samples) - 1)]
return statistics.median(deltas) <= 100
```

A median delta of ≤ 100 is characteristic of a global counter; large irregular deltas indicate random assignment. Only peers classified as sequential have their subsequent IP IDs monitored for anomalous jumps; flagging random-ID peers would produce pure noise. Modular arithmetic handles the 16-bit wrap-around at 65535⟶0.

4.4.3 Route Tracking and ASN Enrichment

`RouteTracker` runs traceroute to a target address and maintains a per-destination baseline hop sequence. The subprocess is invoked with `-n` (no DNS reverse lookups, for speed), `-q 1` (one probe per hop rather than the default three), and a timeout derived from the maximum hop count:

```
traceroute -n -q 1 -w 5 -m 30 <destination>
```

The output parser deduplicates the three probe lines traceroute emits by default using a `seen_hops` set, taking only the first responding IP per hop number. Silent hops (`* * *`) are recorded as `RouteHop` entries with `ip=None`.

`_compare()` evaluates three change types:

- **Hop count change**: total hop count differs.
- **Hop sequence change**: Jaccard similarity of responding-IP sets falls below 0.7, a tighter threshold than the port Jaccard (0.6) because legitimate route changes are less frequent than port profile variations.
- **New ASN in path**: an autonomous system number appears in the observed path that was not present in the baseline.

Any of these fires `route_changed` (+1.0 for hop changes, +1.5 for new ASN).

ASN resolution uses Team Cymru's DNS-based BGP origin service [10]. The query format is a reverse-octet DNS TXT lookup: `<reversed-octets>.origin.asn.cymru.com`. Two resolvers are tried in order: `dns.resolver` (fast, requires the `dnspython` package) and a raw socket whois connection to `whois.cymru.com:43` (no external dependency). Results are cached in-process in `_ASN_CACHE` to avoid repeated lookups for the same hop IP across ticks. RFC-1918 and loopback addresses are skipped — private-network hops have no public ASN.

A new ASN appearing in a traceroute path is a high-confidence signal for a BGP prefix hijack or an MITM device inserted as a routing hop: both introduce an autonomous system boundary not present in the established baseline.

4.5 Fleet Correlation

Per-peer fingerprint comparison detects that *a* device has changed; it cannot determine whether multiple devices changed *simultaneously*. `FleetCorrelator` addresses this gap by examining the full event log across all peers within a single tick window and firing coordinated-pattern detections when several peers show the same anomaly type concurrently. It runs after all per-peer ingestion is complete and before `SuspiciousAgent`, so that investigation prompts already contain fleet context.

4.5.1 Tick Window and Event Collection

`analyse()` opens a window from `peer_store.last_tick_at` to the current time. On the first tick `last_tick_at` is `None` and the method returns immediately with no events: there is no prior baseline to define a window boundary.

Event collection builds a `recent` mapping of peer ID to the list of event type strings fired since `window_start`:

```
recent: dict[str, list[str]] = {}
for peer_id, peer in self._store.peers.items():
    events_in_window = [
        e.event for e in peer.identity_history
        if e.timestamp >= window_start
    ]
    if events_in_window:
        recent[peer_id] = events_in_window
```

Using `last_tick_at` as the boundary rather than `now() - scan_interval` is deliberate. `last_tick_at` is persisted in `peer_store.json` at the end of every tick; on daemon restart it is

reloaded from the snapshot. Events that fired before the restart have timestamps earlier than the loaded `last_tick_at` and are therefore excluded from the next tick's window. A wall-clock interval would have no such anchor and would re-process historical events on every restart.

4.5.2 Pattern Matching, Boost Cap, and Fleet Context

The pattern registry is a flat list of 4-tuples: name, trigger event set, min-peers config attribute, and base boost:

```
_PATTERNS = [
    ("arp_poisoning",    frozenset({"arp_spoofing_detected"}),
"fleet_arp_min_peers", 2.0),
    ("identity_sweep",   frozenset({"identity_conflict_detected",
"full_identity_shift"}), "fleet_identity_min_peers", 2.0),
    ("route_shift",      frozenset({"route_changed"}), "fleet_route_min_peers",
1.5),
    ("os_normalisation", frozenset({"os_family_changed"}), "fleet_os_min_peers",
1.5),
    ("ttl_shift",        frozenset({"ttl_deviation"}), "fleet_ttl_min_peers", 1.5),
    ("service_sweep",    frozenset({"service_type_changed"}),
"fleet_service_min_peers", 1.0),
]
```

A peer "hits" a pattern if it fired at least one event in the trigger set during the current window. A pattern fires if the number of hitting peers meets or exceeds its `min_peers` threshold. The `frozenset` semantics produce OR matching within a pattern: `identity_sweep` triggers on either `full_identity_shift` or `identity_conflict_detected` because a fleet-wide device-substitution attack may produce either event depending on whether the attacker clones the victim's MAC. Min-peer thresholds differ by pattern: `arp_poisoning` requires only 2 peers (an attacker targeting a gateway and one host is sufficient), while `service_sweep` requires 4 (a scan-wide service change is only suspicious at scale). All thresholds are configurable so operators can tune them for their subnet size.

Boost application is capped per peer per tick. A `boost_applied` dict tracks cumulative boost received this tick; each pattern's contribution is clipped to the remaining headroom:

```
headroom = max(0.0, cap - boost_applied.get(pid, 0.0))
actual_boost = min(boost, headroom)
```

The default cap is 4.0. Without it, a peer matching all six patterns in a single tick would accumulate 9.5 in fleet boosts alone, immediately triggering a remediation block on what may be a coincidental co-occurrence of routine events. The cap ensures fleet boosts amplify existing per-peer evidence rather than substitute for it. Each boost is recorded as a `fleet_correlation_boost` event on the peer via `PeerStore.add_suspicion()`, preserving a full audit trail of the scoring decision.

`FleetEvent` objects serve two purposes after `analyse()` returns. The daemon appends each to `fleet_alerts.jsonl` for persistent audit. All events are also passed to `SuspiciousAgent.investigate_all(fleet_events=...)` so the LLM prompt includes the coordi-

nation context: pattern name, number of peers involved, and the list of affected IPs. Without this injection, each peer's investigation would be independent and the simultaneous co-occurrence, the very signal that distinguishes a coordinated attack from a coincidental configuration change, would be invisible to the agent.

4.6 LLM-Assisted Investigation

`SuspiciousAgent` is invoked after fleet correlation completes. It runs a four-step investigation pipeline for each peer whose suspicion score meets or exceeds `suspicion_threshold`, produces a structured JSON report, and optionally feeds cryptographic anchor results back into `PeerStore`. The LLM is strictly in the triage path: it produces a severity verdict and recommendations, but it never issues system calls and is never in the authorisation path for remediation.

4.6.1 Investigation Pipeline

`investigate_all()` selects peers with `score >= threshold`, builds a per-peer lookup of any `FleetEvent` objects that include that peer, and calls `investigate()` for each. A single peer investigation proceeds in four steps:

1. `_analyse()` — format peer context into a prompt, invoke the LLM, parse the JSON response into an `AgentDecision`.
2. `_build_auto_identity_checks()` — append `ssh_hostkey` and `ssl_cert` scan recommendations for any peer with SSH or HTTPS ports open, regardless of what the LLM recommended.
3. `_execute_scans()` — execute each recommended scan type; some results feed back into `PeerStore`.
4. `_write_report()` — serialise the full `InvestigationReport` to `reports/investigation_<mac>_<timestamp>.json`.

4.6.2 Prompt Design and Injection Prevention

`_format_peer_context()` constructs the human-turn message passed to the LLM. The included fields are: MAC address, IP set, suspicion score, OS fields from the latest `NormalisedData`, all OS candidates with accuracy scores, device vendor string, open ports, current services dict, per-port `known_services` history, and the timestamped event history with structured detail dicts. When fleet events are present, a labelled `FLEET CONTEXT` section is appended with the pattern name, affected peer count and IPs, and the window timestamps.

What the prompt deliberately excludes is equally important. Raw packet payloads, nmap service banner strings, DNS hostnames, and any other unstructured data originating from the network are not forwarded. The `details` dicts in `IdentityEvent` records contain only structured fields (`port=22`, `old_service="ssh"`, `new_service="http"`) not raw wire data. This is the primary mitigation for indirect prompt injection [13]: an attacker cannot craft a service banner or hostname that alters the LLM's severity verdict, because those strings never appear in the prompt.

The system prompt in `prompts/suspicious_agent.txt` includes a Fleet Context Guide section that explicitly instructs the model to increase severity when a fleet pattern is present and to name the pattern in its explanation. This structured instruction, rather than relying on the LLM to infer the significance of fleet data from raw peer lists, produces more consistent severity calibration across different model versions.

4.6.3 LLM Integration, Output Schema, and Fallback

The LLM client is initialised with:

```
self.llm = init_chat_model(
    model, model_provider="ollama", temperature=0, format="json"
)
```

`temperature=0` produces deterministic outputs for identical inputs, making investigations reproducible and making it easier to distinguish model-introduced variance from genuine signal changes. `format="json"` enables Ollama's grammar-constrained decoding, which forces the model to emit syntactically valid JSON regardless of how verbose its base response tendency is. `_strip_code_fence()` handles the residual case where a model wraps its JSON in a markdown code block despite the constraint.

The expected response is validated into `AgentDecision`:

```
class AgentDecision(BaseModel):
    explanation: str
    severity: str          # "low" | "medium" | "high"
    recommended_scans: list[ScanRecommendation]
    recommended_actions: list[str]
```

where each `ScanRecommendation` carries a `type` (one of `nmap`, `traceroute`, `tcpdump`) and a `reason` string.

Any exception during the LLM call, Ollama unavailable, JSON parse failure, network timeout, routes to `_rule_based_fallback()`. The fallback assigns severity by score thresholds (below 4.0 ⟶ low, 4.0–7.0 ⟶ medium, above 7.0 ⟶ high) and inspects the set of fired events to select relevant scan recommendations: a peer that fired `arp_spoofing_detected` receives a tcpdump recommendation. The explanation is prefixed `[Rule-based fallback — LLM unavailable]` so operators can distinguish fallback reports from LLM-produced ones. This ensures FR15 (investigation always completes) regardless of whether Ollama is running.

4.6.4 Automated Follow-up Scans and Cryptographic Anchors

`_build_auto_identity_checks()` adds `ssh_hostkey` and `ssl_cert` recommendations to every investigation that involves a peer with SSH or HTTPS ports open, unconditionally:

```
# Always run cryptographic identity checks for peers with SSH/HTTPS ports open
# — these catch service-mimicry evasion that LLM may not know to request.
auto_checks = self._build_auto_identity_checks(peer, decision.recommended_scans)
all_recs = list(decision.recommended_scans) + auto_checks
```

This is hardcoded detection logic outside the LLM path. A sophisticated attacker can construct a device that perfectly mimics the victim's OS fingerprint, port profile, MAC address, and TTL baseline. What it cannot spoof without the victim's private key is the SSH host key fingerprint. Cryptographic anchors are therefore the highest-confidence identity check available, and deferring their execution to LLM discretion would create an evasion path.

`_execute_scans()` dispatches to five scan types. `nmap` re-scans the target and re-ingests the result into `PeerStore` via `NmapParser` and `add_or_update_peer()`, keeping the stored fingerprint current after investigation. `traceroute` and `tcpdump` run as subprocesses; their output is recorded in the report but does not mutate `PeerStore`. `ssh_hostkey` runs `nmap --script ssh-hostkey`, regex-parses all `SHA256:...` fingerprints from the script output, and calls `peer_store.ingest_ssh_hostkeys()`. On first call the fingerprints are stored as the trusted baseline; on subsequent calls any difference fires `ssh_host_key_changed` (+3.0 to suspicion score). The scan result output is annotated with `[KEY CHANGED on ports [22]]` when a change is detected, making the signal visible in the report without requiring the reader to compare raw nmap XML. `ssl_cert` follows the same pattern using `nmap --script ssl-cert` and `peer_store.ingest_ssl_cert()`, firing `ssl_cert_changed` (+2.0) on fingerprint change.

The choice of Phi-4 14B as the default model reflects a balance between reasoning quality and local inference speed. During development, Mistral 7B and Llama 3.1 8B were evaluated on the same multi-signal investigation prompts; both produced correct severity verdicts on single- event cases but showed weaker reasoning on composite cases where OS change, port drift, and fleet context occurred simultaneously. Phi-4 consistently named the relevant attack pattern and produced fleet-aware explanations. Running locally via Ollama means no investigation data leaves the monitored subnet, satisfying the air-gap requirement and eliminating the latency and availability risk of an external API.

4.7 Autonomous Remediation

`Remediator` is the final stage of each tick. It receives an `InvestigationReport` from `SuspiciousAgent`, evaluates a deterministic guard chain, and — if all guards pass — issues or logs an iptables block. The LLM is not in the authorisation path: the severity field in the report informs guard 3, but a rule check, not the LLM, decides whether to act.

4.7.1 Guard Chain and Enforcement Modes

`evaluate()` applies five guards in sequence; all must pass before a `BlockAction` is constructed and returned:

1. **Never-block whitelist.** Both IP and MAC are checked against `never_block`, a `frozenset` built once at `Remediator.__init__()` from config. Operators should include the gateway and the monitoring host itself. Frozenset membership is O(1) and immutable after construction.
2. **Score floor.** `suspicion_score >= block_confidence_floor` (default 5.0). This is intentionally above the 3.0 investigation threshold: a peer must accumulate further evidence after triggering LLM investigation before becoming block-eligible.
3. **Severity gate.** `severity.lower() == "high"`. The comparison is a deterministic string check against the `InvestigationReport` field; the LLM (or fallback) populates that field, but it cannot bypass the other four guards.
4. **No active block.** `_is_active_block()` scans `blocks.jsonl` for records where `executed=True`, `unblocked_at=None`, and `expires_at > now` for the same IP. Prevents double-blocking a peer whose block has not yet expired.

5. **IP available.** At least one IP must be resolvable from the peer record; volatile peers with no confirmed IP are skipped.

When all guards pass, `evaluate()` constructs a `BlockAction` containing the two iptables command sequences and an expiry timestamp:

```
block_cmds = [
    ["iptables", "-I", "INPUT",  "-s", ip, "-j", "DROP"],
    ["iptables", "-I", "OUTPUT", "-d", ip, "-j", "DROP"],
]
expires_at = now + timedelta(hours=cfg.block_ttl_hours)
```

`-I` (insert) places the DROP rule at the head of the chain, preventing a downstream ACCEPT rule from overriding it. Both INPUT and OUTPUT chains are targeted to block inbound traffic from the suspect device and prevent any outbound exfiltration to it.

`act()` dispatches to one of three mode handlers. Root privilege is checked once at `__init__()`: if `enforce` mode is requested but `os.geteuid() != 0`, the mode is silently downgraded to `dry_run` with an error log entry. The downgrade is unconditional — not retried per call — so misconfigured deployments fail safely rather than intermittently.

- **`dry_run`** (default): logs the would-be iptables commands and appends a `BlockRecord(executed=False)` to `blocks.jsonl`. No system state is modified.
- **`confirm`**: prints the block candidate — IP, score, reason, expiry, and the exact commands — to stdout and calls `input("Execute? [y/N]")`. `EOFError` and `KeyboardInterrupt` both default to `"n"`, ensuring that non-interactive sessions (SSH without a TTY, piped input) never accidentally confirm a block. A `"y"` response delegates to `_enforce()`.
- **`enforce`**: runs the iptables commands via `subprocess.run()`, appends `BlockRecord(executed=True)` on success or `BlockRecord(executed=False)` on failure.

The `reason` field stored in `BlockRecord` is built from the last five identity events on the peer, falling back to the first 120 characters of the LLM explanation. It is human-readable context for the audit log and is never passed to iptables.

4.7.2 Block TTL, Audit Log, and Atomic Rewrite

`unblock_expired()` is called at the end of each daemon tick, after `act()`. It reads all records from `blocks.jsonl`, identifies those where `executed=True`, `unblocked_at=None`, and `expires_at <= now`, runs the stored `unblock_cmds`, stamps `unblocked_at = now`, and rewrites the file.

The JSONL file uses two write strategies depending on the operation. Appending a new record uses `_append_record()` — a single-line write that cannot produce a partial file. Updating existing records (to set `unblocked_at`) requires a full rewrite; `_rewrite_records()` performs this atomically:

```
with tempfile.NamedTemporaryFile(mode="w", dir=dir_, delete=False, suffix=".tmp")
as tmp:
    for record in records:
```

```
        tmp.write(record.model_dump_json() + "\n")
tmp_path.replace(self._blocks_path)
```

`Path.replace()` is an atomic rename on POSIX systems: if the daemon is terminated between the write and the rename, `blocks.jsonl` is left intact and the orphaned `.tmp` file is harmless. Without this guarantee, a crash during rewrite could truncate the audit log, causing `_is_active_block()` to miss an active block entry on the next tick and issue a duplicate block.

Together, the guard chain and TTL expiry implement the complete block lifecycle: a peer transitions from eligible → blocked (or dry-run logged) → auto-unblocked at TTL expiry, with every decision — including declined `confirm` prompts and downgraded `dry_run` records — written to `blocks.jsonl` as a tamper-evident append-only audit trail.

4.8 Configuration and Extensibility

All thresholds, weights, and operational parameters are consolidated in a single `PeerWatchConfig` Pydantic model defined in `config.py`. Every field carries a typed default and an inline `description` kwarg that doubles as schema documentation:

```
port_jaccard_threshold: float = Field(
    default=0.6,
    description="Minimum Jaccard similarity between port sets before flagging
drift",
)
remediation_mode: Literal["dry_run", "confirm", "enforce"] = Field(
    default="dry_run",
    description="dry_run: log only | confirm: prompt | enforce: execute (requires
root)",
)
```

`PeerWatchConfig` inherits from `BaseModel`, not `BaseSettings`: there is no environment variable resolution. Configuration is always loaded from an explicit JSON file path via `load_config()`, which handles three cases — `None` or missing file returns a fresh instance with all defaults; a present file is parsed with `json.load()` and validated through `PeerWatchConfig(**data)`. Pydantic validation runs at construction time, so a misconfigured file (wrong type, `remediation_mode: "auto"`, negative threshold) raises a `ValidationError` before the daemon starts rather than at the point of first use. The `Literal` constraint on `remediation_mode` is a particularly useful type-level guard: it makes three semantically distinct operating modes an exhaustive, validated enum rather than a convention-dependent string.

All components receive the config instance as a constructor argument; there is no global config state. `PeerStore`, `FleetCorrelator`, `Remediator`, and `SuspiciousAgent` each hold a reference to the same `PeerWatchConfig` object, which means tuning a threshold in `config.json` and restarting the daemon propagates the change everywhere without modifying any detection logic.

Adding a new passive or active detection signal follows a three-step extension path. First, define the event name string and call `peer.record_event()` with it at the detection site in `peer_store.py`.

Second, add a typed `float` field to `PeerWatchConfig` with a conservative default and description. Third, add a scoring case in `_check_incoming_fingerprint()` (or the relevant `ingest_*()` method) that reads the new weight from `self._cfg`. No changes to `FleetCorrelator` or `SuspiciousAgent` are required unless the new signal warrants a fleet pattern — in which case a single tuple is added to the `_PATTERNS` list in `fleet_correlator.py` alongside the corresponding `int` min-peers field in config. This separation of signal definitions, weights, and pipeline orchestration keeps the system extensible without requiring broad refactors.

REFERENCES

[1] Manos Antonakakis, Tim April, Michael Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, and others. 2017. Understanding the Mirai Botnet. In *Proceedings of the 26th USENIX Security Symposium*, 2017. 1093–1110.

[2] Stefan Axelsson. 2000. The Base-Rate Fallacy and the Difficulty of Intrusion Detection. *ACM Transactions on Information and System Security* 3, 3 (2000), 186–205.

[3] Philippe Biondi. 2003. Scapy – Packet Manipulation Library.

[4] Franco Callegati, Walter Cerroni, and Marco Ramilli. 2009. Man-in-the-Middle Attack to the HTTPS Protocol. *IEEE Security & Privacy* 7, 1 (2009), 78–81.

[5] Harrison Chase and others. 2022. LangChain: Building Applications with Large Language Models.

[6] Cisco Systems. 2020. Cisco Annual Internet Report (2018–2023) White Paper.

[7] Cisco Systems. 2024. Cisco Secure Network Analytics.

[8] Samuel Colvin and others. 2024. Pydantic — Data Validation Using Python Type Hints.

[9] Darktrace. 2024. Darktrace Enterprise Immune System.

[10] Zakir Durumeric, David Adrian, Ariana Mirian, Michael Bailey, and J. Alex Halderman. 2015. A Search Engine Backed by Internet-Wide Scanning. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015. 542–553.

[11] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. 2013. ZMap: Fast Internet-Wide Scanning and Its Security Applications. In *Proceedings of the 22nd USENIX Security Symposium*, 2013. 605–620.

[12] Robert David Graham. 2013. masscan: Mass IP Port Scanner.

[13] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Appli-

cations with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

[14] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. Nodoze: Combatting threat alert fatigue with automated provenance triage. In *network and distributed systems security symposium*, 2019.

[15] Van Jacobson, Craig Leres, and Steven McCanne. 1994. libpcap – Packet Capture Library.

[16] Craig Leres and Van Jacobson. 1992. arpwatch – Ethernet/IP Address Pairings Monitor. Retrieved from https://ee.lbl.gov/

[17] Gordon Fyodor Lyon. 2008. *Nmap Network Scanning*. Insecure.Com LLC. Retrieved from https://nmap.org/book/

[18] Keyu Man, Zhiyun Qian, Zhongjie Wang, Xiaofeng Zheng, and Youjun Huang. 2020. DNS Cache Poisoning Attack: Resurrections with Side Channels. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020.

[19] Markus Miettinen, Samuel Marchal, Ibbad Hafeez, N. Asokan, Ahmad-Reza Sadeghi, and Sasu Tarkoma. 2017. IoT Sentinel: Automated Device-Type Identification for Security Enforcement in IoT. In *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2017. 2177–2184.

[20] MITRE Corporation. 2024. MITRE ATT&CK Enterprise Tactics, Techniques and Procedures.

[21] Ollama. 2024. Ollama – Run Large Language Models Locally.

[22] Open Information Security Foundation. 2024. Suricata Network IDS, IPS and NSM Engine.

[23] KimiNewt Ouri. 2014. PyShark – Python Packet Parsing using tshark.

[24] Vern Paxson. 1998. Bro: A System for Detecting Network Intruders in Real-Time. In *Proceedings of the 7th USENIX Security Symposium*, 1998.

[25] David C. Plummer. 1982. *An Ethernet Address Resolution Protocol.* Retrieved from https://www.rfc-editor.org/rfc/rfc826

[26] Fauzan Prasetyo Eka Putra, Ubaidi Ubaidi, Alief Badrit Tamam, and Reynal Widya Efendi. 2024. Implementation And Simulation Of Dynamic Arp Inspection In Cisco Packet Tracer For Network Security. *Brilliance: Research of Artificial Intelligence* 4, 1 (2024), 340–347.

[27] Qualys Threat Research Unit. 2024. regreSSHion: Remote Unauthenticated Code Execution Vulnerability in OpenSSH Server (CVE-2024-6387).

[28] Martin Roesch. 1999. Snort: Lightweight Intrusion Detection for Networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, 1999. 229–238.

[29] Dug Song and others. 2004. dpkt – Fast, Simple Packet Creation and Parsing.

[30] US-CERT. 2017. TA17-181A: Petya/NotPetya Ransomware.

[31] Wireshark Foundation. 2024. Wireshark Network Protocol Analyser.

[32] Michal Zalewski. 2012. p0f v3 – Passive OS Fingerprinter.

E ARTIFACT APPENDIX

In this section we show how to reproduce our findings.